ZIPPER

Complete Product & System Documentation

An end-to-end description of the system as it exists today

Documentation phase
Phase 0 — Discovery

Version
1.0

Date
3 October 2026

Scope
Android application, CRM, backend, Cloud Functions, Firestore, MongoDB

Method
Source-evidenced. No assumptions. Unknowns recorded, not filled.

Status
Describes. Does not redesign.

Phase 1 (gap validation) and Phase 2 (business-rule confirmation) follow this document. Nothing here is a change proposal.

Contents

How to read this document **6**
- The four classifications . . . 6
- Two truths, kept apart . . . 6
- Where the money is concerned . . . 6
- What this document does not do . . . 6

1. Executive Summary **7**
- What ZIPPER is . . . 7
- Who uses ZIPPER . . . 7
- How money enters and moves . . . 7
- What is automatic . . . 8
- The four most significant findings . . . 8
- What is implemented, what is not . . . 8

2. Product Overview **9**
- 2.1 The proposition . . . 9
- 2.2 Why agencies exist . . . 9
- 2.3 Why coin sellers exist . . . 9
- 2.4 The two currencies, and the third . . . 9
- 2.5 The shape of the organization . . . 10

3. System Architecture **11**
- 3.1 The four codebases . . . 11
- 3.2 Which system owns which truth . . . 11
- 3.3 How the systems actually talk . . . 11
- 3.4 Authentication . . . 12
- 3.5 Regions and time . . . 12

4. Organization and Governance **13**
- 4.1 The Observer — `314zipper` . . . 13
- 4.2 Owners . . . 13
- 4.3 The six departments . . . 14
- 4.4 The twenty permissions . . . 14

5. Roles and Permissions **16**
- 5.1 The complete role list . . . 16
- 5.2 Three things that are not roles . . . 16
- 5.3 Role combinations . . . 16
- 5.4 Permission matrix — who may do what . . . 17
- 5.5 How a screen refuses someone . . . 17

6. User Lifecycle **18**
- 6.1 The states . . . 18
- 6.2 Registration and onboarding . . . 18
- 6.3 Deletion and recovery . . . 19
- 6.4 Suspension and ban, from the person’s side . . . 19

7. Host Lifecycle **20**
- 7.1 Becoming a host . . . 20

7.2 Verification, and what it buys . . . 20
7.3 Earning . . . 21
7.4 Host levels . . . 21
7.5 Two gaps in the host lifecycle . . . 22

8. Agency Lifecycle **23**
8.1 Two status vocabularies that do not match . . . 23
8.2 The KYC status, and a deliberate asymmetry . . . 23
8.3 Order of operations on application, and why . . . 24
8.4 What the agency can do for itself . . . 24
8.5 Agency suspension . . . 24

9. Sub-agency Lifecycle **25**
9.1 What a sub-agency actually is . . . 25
9.2 How one is created . . . 25
9.3 Who approves one . . . 25
9.4 Depth . . . 26
9.5 Being paid — the two legs . . . 26
9.6 Two more sub-agency findings . . . 26

10. Coin Seller Lifecycle **27**
10.1 Where the balance lives . . . 27
10.2 Public exposure — the seller directory . . . 27
10.3 What the seller does in the app . . . 28

11. Staff Lifecycle **29**

12. The Android Application **30**
12.1 First launch . . . 30
12.2 Permissions . . . 30
12.3 Navigation . . . 30
12.4 Home and discovery . . . 30
12.5 Profile and social . . . 31
12.6 Chat . . . 31
12.7 Wallet screens . . . 31

13. Calls **32**
13.1 The state machine . . . 32
13.2 Who may call whom . . . 32
13.3 Who pays . . . 32
13.4 The call split matrix . . . 32
13.5 Billing arithmetic . . . 33
13.6 The stale-call sweep . . . 33
13.7 Call failure paths . . . 34

14. Live Streaming **35**
14.1 Eligibility . . . 35
14.2 Going live, and what a viewer can do . . . 35
14.3 Viewer tracking — two mechanisms, not reconciled . . . 35
14.4 PK battles . . . 36
14.5 Staff moderation mid-stream . . . 36
14.6 Ending a stream . . . 36

15. Party Rooms **37**
15.1 What a room is . . . 37
15.2 Why seats are callables . . . 37
15.3 Seat state — four fields, deliberately not collapsed . . . 37
15.4 Admin powers . . . 37
15.5 Gifts in a party . . . 38
15.6 Cleanup . . . 38

16. Events **39**

17. Gifting **40**
17.1 The catalogue . . . 40
17.2 The receiver's share . . . 40
17.3 The send sequence . . . 41
17.4 Worked examples . . . 41
17.5 Everything the send writes . . . 42
17.6 Idempotency . . . 42

18. Chance, Lucky Gifting and Player Protection **43**
18.1 What it is . . . 43
18.2 The draw is provably fair . . . 43
18.3 What money moves . . . 43
18.4 Three payout ceilings and a reserve . . . 43
18.5 Player protection . . . 44
18.6 A third-party actor that can move money . . . 44
18.7 Who operates it in the CRM . . . 44
18.8 Status: LIVE or NOT LIVE? . . . 45

19. The Two Financial Systems **46**
19.1 The two systems . . . 46
19.2 They never meet . . . 46
19.3 What that means in practice . . . 46
19.4 Balance mutations with no transaction row at all . . . 47
19.5 Idempotency coverage . . . 47

20. The Commission Engine **48**
20.1 The slab tables . . . 48
20.2 The arithmetic . . . 49
20.3 What "billing" means . . . 49
20.4 Apportionment across hosts . . . 50
20.5 Periods, accrual and payment . . . 50
20.6 Parent and sub-agency money, as implemented . . . 50
20.7 Adjustments and reversals . . . 51
20.8 Provisional figures . . . 51

21. Host Earnings and the Weekly Release **52**
21.1 The cycle . . . 52
21.2 The two gates, and why a skip is not a loss . . . 52
21.3 Converted beans leave the payout pool . . . 53
21.4 Eligibility rules, with their values . . . 53
21.5 The weekly payout report . . . 53

22. Salary **54**
22.1 The flow . . . 54
22.2 What is and is not recorded . . . 54
22.3 Does salary count toward commission billing? . . . 54

23. Withdrawals **56**
23.1 Two parallel mechanisms . . . 56
23.2 Limits, fees and gates . . . 57

24. Recharge and Payments **58**
24.1 How money enters . . . 58
24.2 What does not exist . . . 59
24.3 The recharge retry, and its hazard . . . 59

25. Lending **60**

26. Capital, Expenses, Liabilities and Profit **61**
26.1 Owner capital — neither revenue nor cost . . . 61
26.2 Expenses . . . 61
26.3 Liabilities — three, in three places . . . 61

26.4 Profit — two different formulas . . . 61
26.5 The definitions block . . . 62

27. Financial Reconciliation **63**
27.1 Can every figure be traced? — the fifteen questions . . . 63

28. The CRM **65**
28.1 Two role vocabularies, and why it matters on every screen . . . 65
28.2 Signing in . . . 65
28.3 The money board — five stages . . . 65
28.4 Twenty-two card types . . . 66
28.5 The three queues a person opens first . . . 66
28.6 Finance screens . . . 67
28.7 Operations, support and oversight . . . 67
28.8 Tech and the kill switch . . . 68
28.9 Configuration screens . . . 68
28.10 Department ownership — and where it is unclear . . . 68
28.11 Known navigation and page disagreements . . . 69

29. Cloud Functions and Scheduled Jobs **70**
29.1 The 21 Cloud Functions . . . 70
29.2 Fourteen scheduled jobs . . . 70

30. Notifications **72**
30.1 Push — four messages, all data-only . . . 72
30.2 Email — two separate systems . . . 72
30.3 In-app . . . 73
30.4 SMS and WhatsApp . . . 74

31. Audit **75**
31.1 What a record carries . . . 75
31.2 The vocabulary . . . 75
31.3 What is not audited . . . 75

32. Security **77**
32.1 What the security rules actually govern . . . 77
32.2 The layered model . . . 77
32.3 Identity and session . . . 77
32.4 Sensitive data handling . . . 77
32.5 Where security rules have not been compiled . . . 78

33. Configuration **79**
33.1 Where configuration lives . . . 79
33.2 How a change takes effect . . . 79
33.3 The rate fallbacks . . . 79

34. Reporting **81**
34.1 The conventions that run through every report . . . 81
34.2 The reports . . . 81
34.3 What a partner sees of their own commission . . . 81

35. Integrations and External Actors **83**
35.1 Third-party services . . . 83
35.2 Machine actors . . . 83
35.3 A whole distribution channel: self-hosted updates . . . 83

36. Data Model **85**
36.1 MongoDB — the business of record . . . 85
36.2 Firestore — what the app lives in . . . 85
36.3 Key relationships . . . 86
36.4 Collections declared and never used . . . 86

37. State Machines — Consolidated **87**
37.1 The ticket vocabularies . . . 87

38. Feature Status Classification **89**
A. Implemented and active . . . 89
B. Implemented but disabled, or not reachable . . . 89
C. Partially implemented . . . 89
D. Discovered — business meaning unclear . . . 89
E. Business requirement, implementation not found . . . 89
F. Legacy, dead or unused . . . 90
G. Unknown . . . 90

39. Discovered — Business Meaning Required **91**
39.1 The `chance` subsystem . . . 91
39.2 The gender-based call split . . . 91
39.3 Video cards . . . 91
39.4 The user XP ladder . . . 92
39.5 The three superadmin security questions . . . 92
39.6 Self-hosted application updates . . . 92
39.7 Two profit formulas . . . 92
39.8 `platformCut` mixes two units . . . 92
39.9 Other discovered items . . . 92

40. Gap Register **94**

41. Edge Cases — Documented Behaviour **98**
41.1 People . . . 98
41.2 Money . . . 98
41.3 Calls . . . 99
41.4 Live and party . . . 99
41.5 Chance . . . 99
41.6 Data . . . 99

42. Open Questions **100**
Business . . . 100
Finance . . . 100
Operations . . . 100
Technical . . . 101
Compliance . . . 101
Product . . . 101

43. Phase 1 Investigation Plan **102**

44. Completeness Check **103**
What is NOT covered, and should be . . . 103
PHASE 0 DOCUMENTATION COMPLETE — THIS IS THE CURRENT UNDERSTANDING OF ZIPPER . . . 104

Appendix A — Document Provenance **105**

How to read this document

This is a **description of ZIPPER as it exists today**, reconstructed from the source of all four codebases. It is not a design proposal. Nothing in it recommends a change.

It was written to a strict rule: **every statement is evidenced in the implementation, and nothing is invented.** Where the code did not answer a question, this document says so rather than filling the gap.

The four classifications

Every feature described here carries one of these labels.

Label	Meaning
IMPLEMENTED	The behaviour is present, coherent, and appears to be in live use.
PARTIALLY IMPLEMENTED	Some pieces exist; the end-to-end path is incomplete.
DISCOVERED — BUSINESS MEANING UNCLEAR	The system does this. Why it does it is not stated anywhere.
NOT FOUND	Expected by a business description, but no implementation exists.

Two truths, kept apart

Where a business description and the implementation disagree, both are recorded and **neither is declared the winner**. Those entries appear in the Gap Register (§64) and are resolved in a later phase, not here.

Where the money is concerned

Financial sections state the arithmetic exactly as the code performs it, including rounding direction, and show worked examples with real numbers. Where two parts of the system compute the same figure differently, both are shown.

What this document does not do

It does not rank gaps by business importance, because that ranking is a business decision. It does not propose an architecture. It does not recommend removing anything. It asks no questions until §66.

1. Executive Summary

What ZIPPER is

ZIPPER is a **live social and calling platform** with a managed partner economy. Members of the public install an Android app, buy a virtual currency, and spend it on video calls, live streams, party rooms and gifts. On the receiving side, selected people — **hosts** — earn a second virtual currency from that spending, which converts to real money paid out weekly.

Two kinds of business partner sit between the platform and its people:

- **Agencies** recruit, manage and vouch for hosts, and earn commission on what their hosts cash out.
- **Coin sellers** buy currency wholesale and resell it to app users, taking a margin. They are how money physically enters the business.

A web **CRM** is where every internal decision happens: approvals, moderation, verification, configuration, finance and audit. A six-department staff organization operates it, supervised by multiple owners and a single read-only observer account.

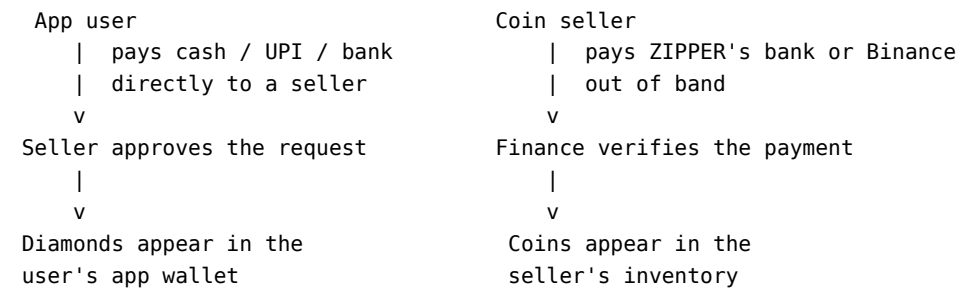
Who uses ZIPPER

Who	Where they work	What they do
App user	Android app	Buys currency, calls, watches, gifts
Host	Android app only	Streams and takes calls; earns; paid weekly
Agency	CRM portal + app dashboard	Recruits hosts; earns commission
Sub-agency	CRM portal	An agency with a parent; paid through it
Coin seller	CRM portal + app panel	Buys currency wholesale, resells for cash
Department staff	CRM	Operate the business in six departments
Owner	CRM	Signs every movement of money
Observer	CRM, read-only	Watches the business; creates owners

How money enters and moves

There is **no payment gateway in ZIPPER.** This is the single most important structural fact about its finances, and it shapes everything else.

Money enters in cash, by bank transfer or Binance, outside the system:



The user then spends **diamonds**. A host receives a share as **beans**. Beans are released a week later, withdrawn as rupees by bank transfer recorded by hand with a UTR reference. An agency earns commission on what its hosts actually cashed out.

What is automatic

Fourteen scheduled jobs run the business without anyone pressing a button: weekly bean release, weekly commission accrual, nightly reconciliation, a per-minute stale-call sweep, daily host-level recalculation, hourly cosmetic expiry, account purging, and seven report and alert sweeps. **Seven of the fourteen appear on no monitoring board.**

The four most significant findings

These are stated here because a reader who stops after this page should still know them.

1. **There are two financial systems that never meet.** The business ledger is in MongoDB. Every app-side movement — every gift, every minute of every call — is written to Firestore and nowhere else. No Cloud Function can write the ledger. A nightly job detects the divergence and corrects nothing. See §41.
2. **The commission engine is progressive, and its own documentation says otherwise.** The implemented rule charges each slice of an agency's billing at its own band's rate, like income tax. Two comments in the codebase still describe a flat rate per band. The executable code and its tests are progressive, and a worked example is confirmed by the owner. See §45.
3. **A parent agency earns nothing from its sub-agencies' production.** A sub-agency has its own, poorer slab table and its own billing; the parent is a payment conduit. No override, roll-up or parent share exists anywhere in the code. See §45.6.
4. **`chance` is a complete regulated-gambling control plane that nothing calls.** Nearly 7,000 lines implementing commit-reveal draws, signed configuration versions, RTP monitoring, payout ceilings and a full responsible-gambling system. The Android app does not reference it. See §29.

What is implemented, what is not

Area	State
Identity, onboarding, moderation	Implemented and in use
Calls, live, party rooms, gifting	Implemented
Commission, host payouts, lending, capital	Implemented
Reports, alerts, audit, scheduled jobs	Implemented
Games tab in the app	Placeholder — "Games — Phase 9"
Events	A feature flag that gates nothing
Payment gateway, webhooks	Not found
Chance / lucky gifting	Server complete, no client
Host transfer between agencies	Not found
Agency-suspension cascade	Not found
Payout hold	Read in four places, written by nothing

2. Product Overview

2.1 The proposition

A person installs ZIPPER and finds a grid of other people. They can look at a profile, send a message, place a video call, watch a live stream, or join a party room. Calls and gifts cost **diamonds**, which must be bought. The person on the other side of a call or a gift, if they are a **host**, earns **beans**.

The platform makes its money on the spread: a host receives a percentage of what the user spent, and ZIPPER keeps the rest. That percentage is configurable per gift and, for calls, is drawn from a table indexed by the genders of the two parties.

2.2 Why agencies exist

ZIPPER does not recruit hosts directly. **There is no apply-to-be-a-host path anywhere in the product.** The only way to become a host is to be given a referral code by an agency and type it into the app.

An agency therefore performs the recruiting, vouching and day-to-day management of its hosts, and uploads the identity documents that make a host payable. In exchange it earns commission, computed on the total its hosts cash out in a period and banded progressively.

2.3 Why coin sellers exist

Because there is no payment gateway, somebody has to take the money. A coin seller buys inventory from ZIPPER wholesale, paying into the company's own bank account or Binance address, and then sells to app users at retail, collecting cash, UPI or bank transfers directly and keeping the margin.

A user who wants diamonds is shown a **directory of verified sellers** with a phone number and a payment QR code. They pay the seller out of band, send a screenshot, and the seller approves the request in their own app panel — which is the moment diamonds appear.

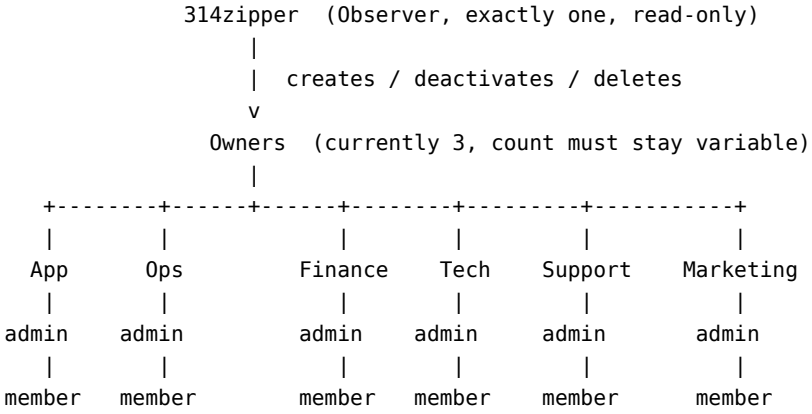
2.4 The two currencies, and the third

Asset	Who holds it	How it is created	How it leaves
Diamonds	App users	Bought from a seller; daily rewards; bean conversion	Spent on gifts, calls, cosmetics
Beans	Hosts	Earned from gifts and calls received	Converted to diamonds, or withdrawn as rupees
Coins	Coin sellers	Bought wholesale, or lent on credit	Consumed when fulfilling a user's recharge

Beans and diamonds are not interchangeable at par. The published rate is **20 beans to 1 diamond**, and the code states the reason plainly: without that friction, gift → convert → re-gift would be a closed loop minting diamonds nobody paid for.

Commission is a fourth ledgered balance, held by agencies and denominated in paise.

2.5 The shape of the organization



Partners sit outside the staff tree entirely:

```
Agency  -->  Sub-agency  -->  Host  (an app account, never a CRM login)
Coin seller
App user
```

3. System Architecture

3.1 The four codebases

Codebase	Technology	What it is
`zipper-app`	Android, Kotlin, Jetpack Compose	The consumer product. 49 screens.
`zipper-crm`	Next.js, TypeScript	The staff and partner web application. 78 routes.
`functions`	Firebase Cloud Functions, TypeScript	21 deployed functions: triggers, callables, schedules.
`backend`	Python, FastAPI	The business API. 15 routers, 23 services. Owns money, staff and audit.

3.2 Which system owns which truth

This is the architectural question that matters most, and the answer is split.

Truth	Owned by	Notes
Who a person is (profile, balances the app reads)	**Firestore**	The Android app reads and writes it directly.
Staff, owners, audit, approval cards	**MongoDB**	Written only by the FastAPI backend.
The business ledger	**MongoDB**	`apply_movement` is the only writer.
App-side value movement	**Firestore**	Gifts, calls, conversions, rewards. Never reaches Mongo.
Configuration	**Firestore** (`appConfig/*`)	Read by all four codebases.
Commission balances	**MongoDB**, mirrored to Firestore	Mongo authoritative; the Firestore field is a delta-applied replica.

3.3 How the systems actually talk

```
Android app
   |  Firestore SDK (direct reads and writes, governed by security rules)
   |  Callable functions (every money movement)
   v
Firestore  <------ Cloud Functions (triggers, Admin SDK, rules bypassed)
   ^   |
   |   |  read by
   |   v
FastAPI backend  <----> MongoDB  (ledger, staff, cards, audit, partners)
   ^
   |  HTTP + session
   v
CRM (Next.js)  -- also reads Firestore directly for profiles and config
```

Three observations follow from this diagram, each evidenced in the code:

1. **The CRM is a dual-mode client.** It reads partner and user profiles straight from Firestore, and takes every decision that must be audited through the backend API. Several screens hold two identities at once — a Firestore CRM role and a backend API role — and gate on different ones.

2. **Cloud Functions cannot write the business ledger.** They are TypeScript running next to Firestore; the ledger is a Mongo collection written by one Python function. There is no bridge. The code says so in its own words: *"The app spends diamonds through Cloud Functions that Mongo never hears about."*
3. **The Admin SDK bypasses security rules.** Everything a Cloud Function or the backend writes to Firestore is unconstrained by `firestore.rules`. The rules therefore govern the Android app and the CRM browser only — a fact that determines what a rule is actually worth.

3.4 Authentication

Surface	Methods
Android app	Phone + OTP, Google, Guest, quick-login (device secret). **No email/password.**
CRM staff	Email + password, Google (linked to an existing record only)
CRM observer	One-time code emailed to a separate mailbox. No password exists.
CRM partner	Email + password, created during application
Backend service	`X-Service-Key` header, three narrowly-scoped credentials

A **superadmin** faces an extra gate every login: three personal questions, three attempts, then a forced sign-out.

3.5 Regions and time

Cloud Functions run in `asia-south1`. The business calendar is **Asia/Kolkata** — all fourteen scheduled jobs are expressed in IST, and daily and weekly keys are cut on IST boundaries. **The commission window is the exception: it is computed in UTC.** An ISO week for commission runs 00:00 UTC Monday to 00:00 UTC the following Monday, while the bean-release job that feeds it runs at 00:30 IST.

4. Organization and Governance

4.1 The Observer — `314zipper`

IMPLEMENTED, and built unlike anything else in the system.

There is exactly one observer account. Its purpose is oversight, and its defining property is that **it cannot write anything, anywhere**.

The implementation achieves this in an unusual way: **the observer has no `crmUsers` document at all.** The code that mirrors staff records into Firestore explicitly refuses to create one, and states why — the CRM's role vocabulary has no read-only tier, so any document the observer were given would grant write access on every Firestore-backed page.

Property	Implementation
Count	Exactly one. `314zipper@gmail.com` by default.
Sign-in	A one-time code, emailed to a mailbox that is **not** the observer's own address.
Password	None exists.
Google sign-in	Refused on every path, by address.
Firestore record	None, by design.
Permissions held	An empty set.
Write capability	`can_write` returns false. Enforced in the API independently of the UI.

What the observer can read: a hard allowlist of eleven route prefixes — Waiting on you, Money Board, Weekly Report, Reports, Finance, Alerts, Scheduled Jobs, Email Delivery, Audit Log, Statements, Team Access. Anything else renders *"Not available to the observer."* The observer gets an entirely separate sidebar, footer-badged "Read only".

Two deliberate decisions are documented inline. The whole `/finance` prefix is **included**, because hiding the money from the one role that exists to watch it would defeat the purpose. And `/tech/email` and `/tech/jobs` are listed individually rather than `/tech`, because prefix matching would otherwise open Content, Payments and the kill switch.

What the observer can do that nobody else can: create an owner. `assignable_roles` returns `[OWNER]` for the observer and for no one else.

> **GAP-002** — Business rule: *only the Observer may create, deactivate or delete an Owner.* Implementation: the creation half exists. **No path to deactivate or delete an owner exists anywhere in the system.**

4.2 Owners

IMPLEMENTED. Currently three; the count is not hardcoded anywhere.

An owner is spelled `owner` in the backend and `superadmin` in the CRM — one role, two vocabularies. The mapping is explicit, and the reverse direction is deliberately refused: a repair script may not translate `superadmin` back into `owner`, because that would let an automated run mint an account that can sign money.

An owner holds **all twenty permissions**. More importantly, an owner is the only role that can:

- **Sign a money card.** Nothing moves without an owner's signature.

- **Execute** a balance card, a commission payout, a coin purchase, a sale, or a capital settlement.
- Approve anything in the Approval Queue.
- Open any of the seven App Configuration pages.
- Activate a signed `chance` configuration, or resume a suspended mechanic.
- Release a suppressed alert.
- Send an email broadcast to every app user.

The separation-of-duties rules

These are enforced in the backend and mirrored in the browser so the reason appears before the click, not after.

Rule	Why it exists
You cannot verify a card you raised	One person must not be the whole chain
You cannot sign a card you raised	*"You raised this card. Another owner must sign it."*
You cannot sign a card you verified	Two different people, two different acts
You cannot sign a card that pays you	Added because an owner could otherwise legally sign their own capital contribution
Nothing skips Under Review, including an owner's own card	

4.3 The six departments

IMPLEMENTED exactly as six, with two tiers each.

The number twelve that appears in some business descriptions is the **staff role count** — six departments multiplied by admin and member — not a department count. The code has exactly six.

Department	What it owns in practice
App	Users and hosts, host verification, live monitoring, the gift catalogue, policies
Operations	Moderation end to end: reports, device bans, appeals. Partner onboarding and KYC approval. Seller sales. Freezing a `chance` player
Finance	Every finance screen, coin purchases, the credit book, economy configuration, weekly payout runs. The only non-owner role that may verify *any* card
Tech	App Control (the kill switch), content, payment methods, scheduled jobs, email delivery
Support	App support tickets, user feedback, read access to user records. Notably **does not** hold the ban permission
Marketing	In-app broadcasts and the email composer. Holds exactly one substantive permission

The reporting rule

Members report to their department's admin; admins report to an owner. Authority is decided by a single predicate that takes the acting person's role and the target's department, and admits **members of that department only** — never a sibling admin, never an owner, never the observer.

> **GAP-003** — Business rule: *if a department admin is deactivated, their members report to an owner until a new admin is appointed, then revert.* Implementation: authority is computed from department membership and tier. **There is no vacancy state.** Nothing detects that a department has no admin, nothing re-points members at an owner, and nothing reverts them.

4.4 The twenty permissions

Held on each staff member's record, written from a single table in the backend and projected into Firestore. The CRM reads the same list back.

Group	Permissions
Users	`can_view_users`, `can_ban_users`, `can_adjust_balance`
Agency & KYC	`can_view_agencies`, `can_approve_kyc`
Sellers	`can_view_sellers`, `can_approve_seller`, `can_coin_operations`

Group	Permissions
Finance	`can_view_finance`, `can_process_payout`
Configuration	`can_economy_config`, `can_commission_slabs`, `can_app_config`
Tickets	`can_create_tickets`, `can_assign_tickets`, `can_close_tickets`
Administration	`can_approve_requests`, `can_manage_team`, `can_monitor_live`
Marketing	`can_send_broadcasts`

Three facts worth stating plainly:

- `can_approve_kyc` and `can_approve_seller` are held by **`ops_admin` and the owner only**. `app_admin` can open the KYC queue and will see no Approve button.
- `can_economy_config` and `can_commission_slabs` are held by **Tech and the owner only** — and the Economy Configuration screen is gated to **Finance and the owner**. Tech holds the permission for a screen it cannot open.
- `can_app_config` is held by `app_admin`, `tech_admin` and `tech_member`, and **all seven App Configuration screens are owner-only.** Three roles hold a permission no screen honours.

5. Roles and Permissions

5.1 The complete role list

Sixteen roles exist in the model.

Tier	Roles	Count
Governance	`observer`, `owner`	2
Staff	`{app, ops, finance, tech, support, marketing}_{admin, member}`	12
Partner	`agency`, `seller`	2

5.2 Three things that are not roles

This is a structural finding worth understanding before reading any permission table.

Concept	How it is actually represented
Host	A boolean, `isHost`, on an app user's record, plus an `agencyId`. Not a role.
App user	A document in `users`. No role entry at all.
Sub-agency	A `parentAgencyId` field on an agency record. Not a separate entity.

A consequence: a host has no CRM identity, cannot sign in to the CRM, and is managed entirely *about* rather than *by*. This matches the business rule.

5.3 Role combinations

IMPLEMENTED for the partner side; enforced by refusal on the governance side.

Combination	Permitted	Evidence
Agency + Coin seller	**Yes**	One login holds a `roles[]` array and a per-capability `roleStatus{}`; a role switcher appears in the CRM
Sub-agency + Coin seller	Yes	Same mechanism
User + Host	Yes	`isHost` on the user record
Host + Agency / Seller	Yes	The partner record names the login
Agency + Sub-agency	**No**	A second capability is created as `type: 'main'`; the code states it is "a new business, not a promotion within the agency hierarchy"
Observer + anything	**No**	The invite function refuses `observer` for every caller
Owner + anything	**No**	Owner is exclusive
Staff + partner	**No**	A staff record carries no partner link field; the directory refuses to write one

A second capability is not self-granted. A partner holding exactly one capability sees an application card, which files a request into the staff queue; nothing is granted until a reviewer approves it with a mandatory reason.

5.4 Permission matrix — who may do what

Compiled from the permission table, the navigation gating and each screen's own constant. "SA" is superadmin/owner.

Action	Observer	Owner	App	Ops	Finance	Tech	Support	Marketing	Member tier	Partner
Read the money board	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	own + team	own only
Raise a money card	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	some
Verify a money card	No	Yes	own team	own team	**any**	own team	own team	own team	**No**	No
Sign a money card	No	**Yes**	No	No	No	No	No	No	No	No
Execute a movement	No	Yes	No	No	No	No	No	No	No	No
Approve KYC	No	Yes	No	**Yes**	No	No	No	No	No	No
Approve a seller	No	Yes	No	**Yes**	No	No	No	No	No	No
Ban or suspend a user	No	Yes	Yes	Yes	No	No	**No**	No	partial	No
End a live stream	No	Yes	No	**Yes**	No	No	No	No	ops only	No
Approve a withdrawal	No	Yes	No	No	**Yes**	No	No	No	No	No
Approve a host salary	No	Yes	No	No	**Yes**	No	No	No	No	No
Edit commission slabs	No	Yes	No	No	**Yes**	propose	No	No	No	No
App configuration	No	**Yes**	No	No	No	No	No	No	No	No
The kill switch	No	Yes	No	No	No	**Yes**	No	No	tech only	No
Send an in-app broadcast	No	Yes	No	No	No	No	No	**Yes**	mkt only	No
Email every app user	No	**Yes**	No	No	No	No	No	No	No	No
Activate a chance config	No	**Yes**	No	No	No	No	No	No	No	No
Freeze a chance player	No	Yes	No	**Yes**	No	No	No	No	No	No
Read the audit log	**all**	all	team	team	team	team	team	team	self	No
Create an owner	**Yes**	No	No	No	No	No	No	No	No	No

5.5 How a screen refuses someone

Three patterns, applied inconsistently across the CRM:

1. **A polite panel** — the dominant pattern, used by most partner, finance, operations and chance screens.
2. **A silent redirect to the dashboard** — used by Analytics, Operations, all App Configuration pages, and App Control.
3. **An inline "Access Denied" block** — used by Broadcasts, the agency register form, the salary screen and the agency dashboard.

Navigation fails closed: until the role record arrives, nothing navigable renders at all.

6. User Lifecycle

IMPLEMENTED.

```
Install
  |
  v
Splash: device ban? session? record? banned? deleting? displaced? profile complete?
  |
  v
Login  (phone OTP | Google | Guest | quick-login)
  |
  v
Profile Setup  (name, gender, DOB 18+, city, languages — all required)
  |
  v
ACTIVE  -----> Home, calls, live, party, gifts, wallet
  |
  +--> SUSPENDED      (a reviewer, or an expiry date; a nightly job lifts it)
  +--> BANNED         (a reviewer; every device the account used is barred too)
  +--> RECOVERY_PENDING  (the person asked to be deleted; 30 days)
            |
            +--> back to ACTIVE  (they signed in and chose "Keep my account")
            +--> PERMANENTLY_DEACTIVATED  (30 days elapsed, or they chose to finish it)
```

6.1 The states

Two parallel fields, one canonical. `accountState` says *why*; **`isBanned` is the single gate** the app, the security rules and the Cloud Functions all read.

State	Set by	Notes
`active`	—	The normal state
`suspended`	A moderator	Also sets `isBanned`, an end date, and who did it
`banned`	A moderator	Clears the suspension fields; bans every device
`recovery_pending`	**The person**	30-day grace, server-set, idempotent
`permanently_deactivated`	The person, or a nightly job	Terminal

Cross-transitions are explicitly refused. Unban refuses a suspended account; resume refuses a banned one. The server names the correct action rather than silently doing the wrong one. The CRM therefore offers only the matching lifting action.

> `accountState` is declared **twice** — five values in TypeScript, three in Python. Nothing reconciles the two declarations; they are kept in step by convention.

6.2 Registration and onboarding

Four ways in, and **no email/password sign-in exists in the app**.

The profile form requires display name, gender, date of birth, a city and at least one language. Three of those choices are consequential:

- **Gender is set once and locked.** The screen says so: *“Set once — it decides your call pricing and can’t be changed later.”* It indexes the call split table on both axes.
- **Date of birth is gated at 18.** A younger date is refused and not stored.
- **Country is never chosen by the person.** It is taken from the device locale, and silently overwritten if location detection succeeds.

Abandoning the form is destructive and says so: *“Signing in already created an account. Going back removes it — the login itself, not just this form.”* Confirming deletes the auth user and the half-written record. **There is no save-and-resume.**

A **valid referral code typed here changes the destination**: the person is told they will be registered as a host and is routed to face verification instead of Home.

6.3 Deletion and recovery

The clearest copy in the product. The person is signed out immediately and disappears from the product, but for 30 days nothing is destroyed. Signing in during the window offers three choices: keep the account exactly as it was, start a new one, or finish it today.

After 30 days a nightly job purges, **at most 50 accounts per run**. The purge releases the identity — phone key, email, phone number, and the auth login — and **deliberately retains seven subcollections**: diamond transactions, bean transactions, daily stats, weekly earnings, warnings, gift showcase and the block list. The reason is written into the code: *“A user tapping ‘delete my account’ is asking to leave, not asking the business to forget that money changed hands.”*

6.4 Suspension and ban, from the person’s side

Both screens keep the person **signed in**, deliberately, because the appeal form writes a record about the signed-in account.

The suspension screen states the distinction outright — *“This is a suspension, not a ban”* — then the staff reason, then one of three honest shapes for the end date: no date on record, a date that has already passed (with an explanation that the lifting job runs on a schedule), or a future date with a day count. It then says *“This phone is not blocked”* and offers a route back to Login for a different account.

The ban screen has **no route off it**, deliberately. A device block adds: *“Every account is blocked on this phone, whoever signs in.”*

The appeal panel is the one thing a banned person can do: free text, up to three screenshots, and a fallback support address. Once sent: *“A person reads every appeal… sending another one will not make it faster.”*

Mid-session bans take effect immediately. A live listener on the user record drops the push token so the phone stops ringing and moves the person to the right screen over whatever they were doing.

7. Host Lifecycle

IMPLEMENTED, with two documented gaps.

```
App user
  |
  |  an agency gives them a referral code  (the only entry)
  v
Code typed in Profile Setup, or on the Join Agency screen
  |
  v
isHost = true, agencyId set, hostVerification opened at pending_capture
  |
  v
Face verification in the app  (3 liveness steps, 3 attempts)
  |
  +--> AI match        --> verified
  +--> below threshold --> pending_manual  --> a reviewer decides --> verified | rejected
  |
  v
VERIFIED HOST  --> may go live, may set own rate, weekly payout releases
  |
  +--> earns beans from gifts and calls
  +--> beans release the following week
  +--> withdraws as rupees (bank transfer, UTR recorded by hand)
  |
  +--> revoke_host  (a reviewer)  --> no longer a host
```

7.1 Becoming a host

There is no application path. The Join Agency screen closes the door explicitly: *"No code? Agencies invite hosts directly — you cannot apply from here."*

Two entries exist:

1. **Code redemption**, in the app. Requires the host application to be `approved`, not merely un-rejected. Refuses a banned account.
2. **A staff grant**, from the CRM. This also opens the verification record, because the capture function refuses when it is missing.

> **Contradiction found:** the Agency Dashboard's empty state tells the person to *"Apply from your profile"*, which contradicts the Join Agency screen and matches no implemented flow.

7.2 Verification, and what it buys

Three states before a verdict: `pending_capture` (seeded, and deliberately **not** reviewable), `pending_manual` (the AI was below threshold, or the agency uploaded no photo), then `verified` or `rejected`.

The app runs a three-step liveness sequence — look at the camera, blink, smile — and also runs an on-device gender classifier, failing the attempt on a mismatch against the declared gender. After three failures the person is sent to Home with reassurance rather than a lockout.

The photographs are treated as **biometric data under India's DPDP Act**. Signed URLs are fetched only when a reviewer opens a case, held in component state, dropped when the case closes, and never logged. The verification record carries the tightest read rule in the system: the host themselves and staff who review — **not the agency**, because an agency that can read the verdict on its own hosts can work out how to get past it.

What verification buys, stated on the app's own profile tab:

Without verification	With it
Cannot go live	Can go live
Earns at the **platform base rate** whatever the agency agreed	Earns at their own rate
Weekly payout is **held**, not lost	Payout releases

The audit entry naming verification approval says it plainly: *"This is the moment someone becomes payable."*

7.3 Earning

Four bean fields on a host's record, each answering a different question:

Field	Question it answers
`beans`	Total balance — spendable and convertible now
`beansAvailable`	Released, and therefore **withdrawable**
`totalBeansEarned`	Lifetime earned
`hostCallEarnings` / `hostGiftEarnings`	Lifetime, split by source

The difference between `beans` and `beansAvailable` is money the host has earned but cannot yet withdraw. The code's reasoning: *"Three honest numbers beat one that sometimes cannot be withdrawn and does not say why."*

7.4 Host levels

A ten-rung ladder scored on **lifetime beans earned from calls and gifts**. The rung does not price a call — it sets the **ceiling on what the host may ask**.

Level	Beans required	Max rate (beans/min)
0	0	700
1	10,000	850
2	100,000	1,000
3	300,000	1,150
4	600,000	1,350
5	1,000,000	1,550
6	2,000,000	1,750
7	4,000,000	1,950
8	8,000,000	2,150
9	16,000,000	2,300
10	32,000,000	2,400

Setting a rate **clamps rather than refuses**, and reports that it clamped. An unverified host cannot set a rate at all. The level is recalculated when a call ends — not per tick, deliberately, because that would be a document read per ten seconds per call — with a nightly sweep at 02:00 IST as the backstop.

7.5 Two gaps in the host lifecycle

GAP-004 — Host transfer between agencies is NOT FOUND. Business rule: a host can move between agencies; App Admin initiates and the Owner decides; historical earnings follow the host while the old agency retains its own record; no double counting. Implementation: no transfer workflow, endpoint, screen or approval path exists. Changing a host's `agencyId` is possible through the generic grant-host action, which neither notifies nor audits it as a transfer, and nothing preserves a dual record.

GAP-005 — `payoutHold` is written by nothing. It is read in four places: the weekly release job blocks on it, two services surface it, and the app renders *"Payouts are on hold for this account."* It appears on the staff-writable allowlist in the security rules. **No CRM screen, no API endpoint and no Cloud Function sets it.** There is no way to put a host on payout hold.

8. Agency Lifecycle

IMPLEMENTED.

```
Application
  |  self-service at /agency-signup  (login created FIRST, then the record)
  |  or staff-filed at /agencies/register
  v
PENDING  ---> partner waits on /agency-pending
  |
  v  reviewed in the /kyc main-agency queue by Ops (can_approve_kyc)
  |
  +--> REJECTED   (reason stays on the record; an applicant told nothing simply reapplies)
  |
  v
APPROVED + isActive
  |
  +--> recruits hosts with its referral code
  +--> creates sub-agencies from its own dashboard
  +--> accrues commission weekly, automatically
  +--> raises salary requests for its hosts
  +--> proposes changes to its own bank details  (live fields do not move until reviewed)
  |
  +--> DEACTIVATED  (isActive false; status unchanged)  --> reactivated
```

8.1 Two status vocabularies that do not match

Store	Values
MongoDB `partners.status`	`pending`, `approved`, `rejected`, **`suspended`**
Firestore `agencies.status`	`pending`, `approved`, `rejected`

A partner suspended in Mongo has **no Firestore vocabulary to express it**. The mirroring function writes the string unconditionally, so `suspended` *is* written into a field whose type does not admit it and which no CRM status badge renders.

`isActive` is independent of `status`. An agency can be `approved` and `isActive: false` at the same time.

8.2 The KYC status, and a deliberate asymmetry

Four values: `pending`, `submitted`, `approved`, `rejected`. Written together with the main status so the two cannot drift. One case is deliberately counter-intuitive:

- Approved **with** documents on file → `kycStatus: approved`
- Approved **without** documents → **left at `pending` on purpose**, because the partner's upload form only renders while the KYC status is `pending` or `rejected`
- Rejected or suspended → `kycStatus: pending`, not `rejected`

8.3 Order of operations on application, and why

The login is created **first**. The reason is recorded inline: creating the login is an authoritative server-side uniqueness test on the email address that writes nothing when it refuses. Then, now signed in, the duplicate query runs — matched against the caller's own verified email, which is the only shape the Firestore read rules permit an applicant. Then the business record. Then the CRM user document.

If the last step fails, **the record is left in place deliberately**, because it is a real application somebody has to decide about. The agency list badges it *"no login attached"* for staff.

8.4 What the agency can do for itself

Surface	Capability
CRM dashboard	See its own commission earned, paid and outstanding, per host and per sub-agency
CRM dashboard	Add a host (files an application), request a salary, mark a paid salary distributed
CRM dashboard	**Create a sub-agency** — the only partner action that creates another partner
CRM dashboard	Propose changes to payout or identity details
App dashboard	See its referral code, host count, sub-agency count, last payout; submit a weekly report

8.5 Agency suspension

> **GAP-006 — The suspension cascade is NOT FOUND.** Business rule: suspending an agency must not lose its hosts or sub-agencies; an owner or App Admin must be notified; hosts and sub-agencies should be transferable to another agency, or to a newly created one; the affected parties must be notified with the new agency's details. Implementation: approve, reject and deactivate each act on **one** record and never query for children. No cascade, no notification to an owner, no transfer workflow, no notice to the host or sub-agency.
>
> The consequence is concrete and asymmetric: nothing happens to the hosts, but a sub-agency payout **requires an approved parent**. Suspending a parent therefore makes every sub-agency's commission **permanently unpayable, with no alert**. The weekly report detects the orphan after the fact — *"there is no main agency to route this commission through, so it cannot be paid"* — which is detection, not prevention.

9. Sub-agency Lifecycle

PARTIALLY IMPLEMENTED. Behaviourally a distinct business; structurally a field on an agency.

9.1 What a sub-agency actually is

There is **no sub-agency collection**. A sub-agency is an `agencies` document carrying a `parentAgencyId`. But its *behaviour* is extensively separate:

Already its own	Shared with a main agency
Commission balance in the ledger, under its own id	The `agencies` collection
Its own slab table, with a higher floor and poorer rates	The `agency` role and every permission
Its own hosts, billed on its own id — **no roll-up to the parent**	The same approval path and KYC permission
Its own review queue, KYC tab and onboarding card	The same login and session scoping
Its own weekly statement	The same registration trigger
Its own two-leg payout mechanism	
Its own KYC document storage path	

9.2 How one is created

Three writers, and **two competing definitions of what makes a sub-agency**.

Route	Who	What it writes
Parent self-service, from the agency dashboard	The parent agency	`type: 'sub'` plus `parentAgencyId`; then creates the sub's login
Staff-filed, `/agencies/register?parentId=…`	App Admin or owner	Same fields, parent from a URL parameter
Normal agency signup	—	**Cannot** create one; hardcodes `type: 'main'`

The server's definition is **two triggers**: `type == 'sub'` **or** a non-empty `parentAgencyId`. Its own comment: *"A parent without `type: 'sub'` is still a sub-agency."* But the agency dashboard's guard tests `type === 'sub'` only. A record carrying a parent but no type is a sub-agency to the server and not to that screen — and a backend test deliberately constructs exactly that document.

Creating the sub's login requires an unusual mechanism: a **secondary Firebase application instance**, because an agency-role user is blocked from writing CRM user documents through the primary one.

9.3 Who approves one

Platform staff, through exactly the same path as a main agency. The parent has no approval role at all. The review queue is separate but the evidence reviewed is the same, and the code says so: *"A sub-agency is reviewed against the same documents as a main agency."*

9.4 Depth

GAP-SA-002 — Depth greater than one level is reachable. Business rule: a sub-agency is exactly one level below an agency and cannot create another sub-agency. Implementation: **no validation, guard or cycle check exists in any of the four codebases.** The only check is one hop — a payout verifies that the named sub's parent is the named parent — and the parent is looked up with no kind constraint, so an approved sub-agency qualifies as a parent.

What prevents it today is the user interface only: two screens hide the "Add Sub-Agency" control when the record's `type` is `sub`. The staff register URL accepts any parent id typed by hand, and the security rules never inspect `parentAgencyId` on create.

As written, a chain A → B → C is creatable and payable leg by leg, nothing reconciles C's money up to A, and a cycle A → B → A is undetected.

9.5 Being paid — the two legs

A sub-agency is never paid directly. The payout has two legs and does not close until both are recorded.

```
Leg 1   ZIPPER pays the main agency
        |  a real ledger movement, debiting the SUB's commission balance
        |  requires a payment reference; owner-signed card
        v
     status: parent_paid       <-- the sub's balance now reads zero
        |
Leg 2   the main agency confirms it passed the money to the sub
        |  NO ledger movement — an assertion, recorded and audited
        v
     status: closed
```

GAP-SA-001 — Leg 2 moves no money, so a sub-agency's receipt is unrecorded. Leg 1 debits the **sub's** balance, so from that moment the sub reads zero while it may not have been paid at all. The system's own words, in the card module: *"the sub sees its balance fall to zero on leg 1 and cannot tell 'Zipper has paid my parent' from 'I have been paid'."* The only record that the sub received anything is the parent's assertion. The question *"has this sub-agency actually been paid?"* cannot be answered from the ledger.

9.6 Two more sub-agency findings

GAP-SA-004 — A dead security clause. The agency read rule contains a comparison between an agency document id and a Firebase auth uid. Those are different identifier spaces, so no document the system writes can satisfy it. Two further rule comments promise sub-agency clauses that are not in the rule bodies.

GAP-SA-005 — `totalSubAgencies` is always zero. It is written as `0` by five creation paths, **incremented by nothing**, and rendered as the agency's sub-agency count on the CRM home page. Separately, a documented `subAgencyPayments` subcollection has no writer, no caller and no security rule.

Not covered by any test: depth and cycles, parent suspension or deletion, the sub-agency security rules, the secondary-app login creation, and the sub-agency counter.

10. Coin Seller Lifecycle

IMPLEMENTED.

```
Application  (self-service /seller-signup, or staff-filed /sellers/register)
  |
  v
PENDING --> reviewed in the /kyc coin-seller queue by Ops
  |
  +--> REJECTED
  v
APPROVED + isApproved + isActive
  |
  v
Appears in the app's VERIFIED RESELLER DIRECTORY  (name, phone, payment QR)
  |
  +--> buys inventory: pays ZIPPER's bank or Binance out of band,
  |    submits a UTR and a screenshot, Finance verifies, an owner executes
  |
  +--> may be LENT inventory on credit, with a due date and a penalty on recovery
  |
  +--> a user pays them directly; the seller approves the recharge in the app;
  |    coins leave their inventory and diamonds appear in the user's wallet
  |
  +--> SUSPENDED / DEACTIVATED  (withdrawn from the public directory)
```

10.1 Where the balance lives

`sellers/{id}.coinBalance` in Firestore. This is the operational figure the app reads and the recharge trigger decrements.

Credit position lives separately, on the Mongo partner record: outstanding, due date, total lent, total written off. **Standing is derived, never stored** — `none`, `current`, `due_soon`, `overdue`, `written_off`, computed from the outstanding amount and the due date. An `overdue` seller is blocked from taking new inventory.

> One derived value is unreachable: `written_off`. The function that computes standing receives only the outstanding amount and the due date, so it cannot see the write-off stamp. A fully written-off seller computes as `none`.

10.2 Public exposure — the seller directory

The directory the app shows is **not** the seller record. It is a separate collection holding exactly seven fields: display name, phone, payment QR URL, description, photo URL, online flag, and the document id.

It is written only by a Cloud Function trigger on the seller record, and is **closed to every client write**. The private seller document — which holds identity documents, coin balance, commission rate, email and the approval trail — is readable by staff and by the seller themselves, and by nobody else.

The reason this exists is recorded in the code: the private collection was once readable by any signed-in account, and because a security rule cannot filter fields and `read` covers both single-document reads and list queries, every seller's identity documents were one directory query away.

10.3 What the seller does in the app

The seller panel shows their coin balance, an **Online/Offline switch** that controls whether buyers can find them, totals, and a queue of pending recharge requests — each showing the buyer, the diamond count, the rupee amount and the buyer's payment screenshot, with Approve and Reject.

Approving is a one-field update to the request. **That single field change is what moves the money** (§44.2).

Restocking is a separate screen showing the margin in plain terms — *"You pay ₹2,290 per lakh. Buyers pay ₹3,145. You collect the difference from them directly."* — then the company's own bank or Binance details, then a UTR field and a screenshot upload.

That screen refuses rather than guesses when the company's details cannot be read: *"Our bank details could not be loaded, so there is nothing safe to show you. Do not transfer to any account you may have used before."*

11. Staff Lifecycle

IMPLEMENTED, with one asymmetry.

```
An existing admin or owner invites an address, naming a role
  |
  |  the roles the form offers are computed on the server,
  |  so it cannot offer one that would be refused
  v
THREE records written together:
   MongoDB staff row  +  Firestore crmUsers document  +  Firebase Auth login
  |
  v
ACTIVE  (born active; a password-reset link is emailed; nobody sets a password for them)
  |
  v
DEACTIVATED  --  is_active: false, the Firestore mirror updated,
                 and the Firebase login disabled
```

There is **no separate invited state**. The row is born active and the account is usable the moment it exists.

Reactivation has no function. Deactivation is one-way in this module.

Because a staff account spans three stores, two dedicated audit actions exist for partial failure — a mirror that could not be written, and a Firebase login that could not be revoked — because a half-created or half-revoked account was previously only an error message the caller could dismiss.

12. The Android Application

49 screens. **IMPLEMENTED**, with eleven documented rough edges collected in §62.

12.1 First launch

The splash screen decides where a person lands, checking eight things in a fixed order: is this handset barred; is anyone signed in; is the stored credential still good; does a user record exist; is the account banned or suspended; is deletion pending; has another phone taken the session; is the profile complete.

Two decisions in that list are worth naming:

- **A device-ban lookup that fails is treated as "not banned"** and the launch continues. The check fails open.
- **Quick-login does not sign anyone in automatically.** A stored device secret earns a button on the Login screen and nothing more. The reason is recorded: a phone left on a desk would otherwise open into a stranger's wallet.

There is no maintenance-mode gate anywhere in the Android app, although the CRM has a maintenance switch.

Force-update is not on the splash. It is a modal layered over the whole application, which waits until the splash has finished deciding, and has no dismiss control when the update is mandatory.

12.2 Permissions

Nothing is requested at install time. Each permission is asked for at the moment it is needed: notifications on reaching the main shell, camera at face verification, camera and microphone behind a rationale screen at Go Live, microphone on entering a party room, and coarse location only if the person taps the detect-location button. Screenshots and screen recording are blocked in release builds.

12.3 Navigation

Five bottom tabs: **Home**, **Live**, **Games**, **Messages**, **Me**.

> **The Games tab is a placeholder.** The decisive line reads `PlaceholderScreen("Games — Phase 9")`. Nothing in the app implements games, and yet the Terms of Service carry a section promising them, the Level screen counts "Lucky Gifts & Games" toward progress, and the diamond wallet has transaction labels for "Game Win Reward" and "Lucky Spin Win".

12.4 Home and discovery

A two-column grid of people, read from the **public profile projection** — never the user record — filtered to listed accounts and ordered by a presence rank. It is a live listener, so cards update as people come and go. The person's own card and anyone they have blocked are removed on the device.

Search is by Zipper ID only — an exact document lookup, not a name search. Filters offer verified-hosts-only, country, language, hobbies and interests; the first three are sent to the server and the last two applied to the already-fetched page.

Empty states are specific rather than generic: *"Nobody's around yet"*, *"Nobody matches these filters"*, *"No one has that ID"*, and at the end of the list *"You've seen everyone online."*

12.5 Profile and social

The Me tab carries three conditional banners — verify your face, join an agency, host earnings — and a grid of tiles: My Bag, My Level, Daily Reward, Likes, Call History, Agency, Seller, Settings, Feedback, Log out.

Another person's profile offers four actions: **Like**, **Call** (overlaid with the live per-minute diamond cost), **Gift**, **Message**. The overflow menu offers Report, with six reasons, and Block, which warns what blocking does.

> **Follow is one-way and live-only.** There is **no Follow button on any profile.** The follow action is called from a live room and nowhere else, and the unfollow function is never called at all — so the follower counts on a profile can only ever rise. A push channel described as *"When someone you follow goes live"* is fed by a feature with no discoverable entry point.

12.6 Chat

Three tabs: Official, Feedback, Chats. **Plain text only** — the composer has no attachment control, although the message model supports images and live invites, and those invites are generated automatically by the Go Live flow. **Messages cost nothing.**

The chat empty state carries a safety line that belongs to the seller economy: *"Messages are private between the two of you. Keep payments to verified resellers only."*

> **Two unconnected ticket systems.** The Messages → Feedback tab reads one collection; the Feedback screen writes to another. A ticket submitted in the app will not appear in the tab meant to show it, and a staff reply writes to a third place again. See §62.

12.7 Wallet screens

Screen	What it does
Diamond Shop	Shows recharge packs with prices. **No in-app payment.** A notice explains that ZIPPER sets the pack price, the person pays a verified reseller directly, and the reseller may add a margin. Tapping a pack opens a reseller chooser with a pre-written message.
Verified resellers	The directory. Each row: name, verified tick, masked phone, a QR button and a Message button. Headed *"Pay only verified resellers. Zipper never asks for OTPs or passwords."*
Diamond wallet	Balance and a transaction list labelled in plain language — "Seller Recharge", "Gift Sent to Priya", "Call — Ravi", "Beans → Diamond".
Beans	Balance, the rate stated as *"20 beans = 1 diamond"*, and a conversion card that accepts only multiples of the rate and previews the result.
Earnings (hosts)	Today, this week, available now, pending release, the release rule stated, blockers, host level and ceiling, and weekly statements each marked Released, Held with a reason, or Pending.
My Bag	Cosmetics: avatar frames, chat bubbles, profile cards, ID frames, entrances, video cards. Rentable with a term; renewal **extends** rather than replaces, so paid days are not discarded.
Daily rewards	A seven-day streak. The schedule is read from configuration but the credited amount is decided on the server, and the screen says so.

13. Calls

IMPLEMENTED. The most precisely specified money path in the system.

13.1 The state machine

Status	Written when	Money
`ringing`	The caller starts it	None
`connected`	The receiver accepts; the clock starts	Billing begins
`ending`	The payer ran out mid-tick	The last affordable tick is already taken
`ended`	Either party hangs up, or the watchdog settles it	Final part-settlement
`declined`	The receiver declined	None
`missed`	The receiver did not answer, or the sweep caught it	None

There is no recovery from `ending` and no re-ring. A second, finer vocabulary records *why*: `user_ended`, `receiver_declined`, `caller_cancelled`, `missed`, `diamonds_exhausted`, `watchdog_timeout`, `no_answer`.

13.2 Who may call whom

Any un-banned user may ring any un-banned user. There is no host-only gate, no verification gate, no presence gate, no gender gate and no age gate on placing a call. The gates that do exist, in order: signed in; a peer who is not yourself; an idempotency key; a replay check; both accounts exist; **the receiver has not blocked the caller**; both sides un-banned; and **the payer can afford one minute**.

The block check is one-directional: a caller who blocked the *receiver* is not stopped.

13.3 Who pays

The call record stores who *earns*; the payer is derived as “the other one”.

Case	Who pays	Price
Exactly one host	**The user pays, the host earns — whichever dialled**	Derived from the earner’s bean income and the split
Two hosts	The caller pays, the receiver earns	From the receiver’s rate
Two users	The caller pays	A flat rate, **1,600 diamonds/min** by default

An **unverified** host earns at the platform base rate — 375 beans/min by default — whatever their level or their agency’s agreement.

13.4 The call split matrix

Reported exactly as implemented, not reinterpreted.

The outer key is the **payer's** gender; the inner key is the **earner's**. The values are the **earner's percentage** of what the payer spends.

Payer ↓ / Earner →	male	female	transgender
male	30%	**50%**	40%
female	**40%**	30%	40%
transgender	40%	40%	40%

Three genders exist. **Anything unrecognised — blank, missing, "other", a guest who never completed setup — resolves to `transgender`**, which is the uniform 40% row and column.

A cell is accepted only if it is a finite number from 1 to 100 inclusive. Anything else, including zero, falls back to the default cell rather than to zero. The table is editable as a 3×3 grid in the CRM, writable by Tech and the owner.

Which gender indexes which axis is decided by who pays, never by who dialled. The code's own examples: a female host calling a male user gives the earner 50%; a male host calling a female user gives the earner 40%. The same direction of dial, a different percentage.

13.5 Billing arithmetic

- **Tick interval: 10 seconds.** Only the paying side's device drives the tick, although either is permitted to.
- **Elapsed time is computed on the server** from the start timestamp. Nothing from the client is used.
- Ticks due is the floor of elapsed seconds over ten; outstanding is ticks due minus the server's own count of ticks already charged. **That count is the duplicate-tick guard** — a replayed tick finds nothing outstanding and charges zero.
- **Per-tick charge** is the ceiling of the per-minute rate over six.
- **Per-tick beans** are the floor of the diamonds charged times the percentage.
- **A partial tick at the end is not charged.**

Worked example

A verified level-0 female host, rate 700 beans/min, called by a male user. The user pays.

Step	Arithmetic	Result
Split cell	payer male × earner female	**50%**
Price per minute	ceiling(700 × 100 / 50)	**1,400 diamonds**
Per tick	ceiling(1,400 × 10 / 60) = ceiling(233.33)	**234 diamonds**
Beans per tick	floor(234 × 50 / 100)	**117 beans**
One minute, six ticks	6 × 234 and 6 × 117	**1,404 out, 702 in**
Platform keeps	1,404 − 702	**702**

The per-tick ceiling makes the payer pay 1,404 rather than the nominal 1,400 — **0.29% above the quoted rate, every minute.** In the unverified case the per-tick floor costs the host 3 beans a minute, 0.8% of the nominal.

A male user paying a male host at base rate is the harshest cell: 1,254 diamonds out, 372 beans in, **platform keeps 70%**.

13.6 The stale-call sweep

Runs **every minute**, with two independent queries.

Case	Condition	Action
A connected call whose client went silent	Last tick more than **2 minutes** ago	Settled through the same path a tap uses, reason `watchdog_timeout`. 2 minutes is 12 missed ticks.

Case	Condition	Action
A call nobody answered	Still ringing after **45 seconds**	Marked `missed` inside a transaction that re-checks the status, so a simultaneous Accept wins cleanly. **No money moves.**

The second case exists for a specific reason, recorded in the code: without it, an abandoned ringing document re-rings days later through the receiver's own query, gets answered, and the watchdog then bills the vanished caller for about twelve ticks of a conversation neither party had.

13.7 Call failure paths

Failure	Behaviour
Payer runs out mid-call	Affordable ticks are charged, the status moves to `ending`, and the app ends the call. If not even one tick is affordable, nothing is charged.
Payer cannot afford the first minute	Refused before the phone rings, with a message that differs by side — the host is told *their caller* has no diamonds.
Host disconnects	The earner is not the ticker. **The payer's device keeps ticking and keeps being charged** until they hang up or the media layer makes the interface react. There is no server-side check that the earner is still present.
Payer disconnects	Ticks stop; two minutes later the watchdog settles at full rate **including the silent gap**.
Server dies after a charge	Each tick is one transaction: all of it commits or none. A lost response with a committed transaction is covered by the tick counter.
Duplicate start	An idempotency claim returns the original call.
Double-tap Accept	Returns "already connected" without resetting the clock.
Hang-up request fails	Ticks **deliberately keep running** until the server agrees, then the person is told: *"You are still connected and still being charged — try End again."*

> **Contradiction:** a caller who gives up on a ringing call files it as a decline. The decline function only honours a "missed" reason from the *receiver*, so the call lands in history as `declined` / `caller_cancelled` while the caller's own screen says MISSED. The two sides disagree about what happened.

14. Live Streaming

IMPLEMENTED, and the one major feature that is **client-owned** rather than server-owned.

There is no start-live or end-live callable anywhere. The room document is created and mutated **directly by the Android client**, and the security rules are the only gate. That asymmetry explains most of the findings below.

14.1 Eligibility

The real gate is a security rule: an active app user who is a host, is a verified host, and is not banned.

- **No level gate.** Nothing checks a host's level before going live.
- **Face detection is not an entry gate.** It starts after the checks and is a *continuity* gate: when the host's face disappears, a 120-second countdown starts and the stream ends at zero. It runs entirely on the host's own device; nothing server-side enforces it. If the detector itself errors, it reports a face as present rather than ending the stream.
- Separately, the media-token function refuses a banned account a token for any channel at all.

Opening a **party room is deliberately the opposite rule**: any signed-in account.

14.2 Going live, and what a viewer can do

A pre-live card takes a title and one of five categories, and carries the warning *"Stay in frame — the live ends automatically if you leave the camera for 2 minutes."*

Starting a stream **auto-messages every online person who has liked the host** with a tappable invite.

The host's rail offers PK, Share, Mute, Flip, Calls off, and picture-in-picture. Tapping the viewer count opens Viewers and Top gifters, with a Remove action per viewer that is explicit about being irreversible.

A viewer can watch, follow, comment, like, and gift. The overflow menu offers hide comments, a stream-quality choice, and Report with six reasons.

Two terminal overlays exist, each specific: *"Removed from this live — the host removed you from this room. You can still watch their other lives."* and *"Live has ended."*

14.3 Viewer tracking — two mechanisms, not reconciled

1. **A bare counter** on the room, incremented on join and decremented on leave. **Any signed-in user may write it.**
2. **A presence document** per viewer, with name, photo and level, owner-scoped by the rules.

Nothing reconciles the two. The kick path reads the presence row first and decrements only if it still exists — the reason recorded inline is that an unconditional decrement double-counted a viewer who had already left, which is exactly when a host reaches for the kick, and enough of those drive the count negative.

Likes are an unbounded, unrecorded counter increment with the failure deliberately swallowed. No per-user cap, no like document, no rate limit.

14.4 PK battles

PARTIALLY IMPLEMENTED — and the rules forbid every write it depends on.

PK is entirely client-side. There is no PK code in the Cloud Functions at all; the only server involvement is the media-token function recognising a shared channel.

The feature has a full interface, a matchmaking queue, a three-minute timer and a score model. **Scoring is by gift quantity, not diamond value** — one 5,000-diamond gift scores 1, ten 1-diamond roses score 10 — and the score is written **by the gifting viewer's own device**, in a second call after the gift has already settled. Nothing on the server writes, validates or even sees a PK score.

The room update rule permits a non-host exactly two fields: the viewer count and the like count. Every PK write fails against it — the score write, the queue claim on another host's room, the two-room accept batch, and the two-room end batch. Because a batch is all-or-nothing, **PK never starts.**

The rule's own comment names what it was closing: *"A viewer could rewrite* `hostUid`*, flip* `isActive` *to kill somebody's stream."* PK was collateral.

14.5 Staff moderation mid-stream

Available to the **owner, Ops admin and Ops member only** — deliberately narrower than who may warn or ban. Every action demands a reason of at least six characters.

Ending a room writes an inactive flag, the end time, who ended it and why, refuses a room that has already ended rather than succeeding silently, and audits after the change. The service is explicit that this is a flag and not a cut stream: the host's app reacts because it watches that flag.

Muting a host writes four fields and audits them. **This is dead on the app side** — the mute field appears in zero Kotlin files. The service says so itself and returns the sentence verbatim to the CRM: *"Until the app honours it, the host is still audible."*

14.6 Ending a stream

The host's app writes exactly three fields: inactive, the end time, and the duration. **Nothing deletes the comments, the gift ticker or the viewer presence documents, and neither counter is zeroed.** Party rooms, by contrast, delete their seats and clear presence on close.

A server-side trigger then updates the public profile projection — live flag, room id, presence rank — and adds the duration to the host's lifetime streamed time, **taken from the number the host's own app wrote**.

The reason the projection exists is recorded: if the feed ranked on the self-declared flag on the user record, setting it would be a permanent top-of-feed slot.

The host's summary card shows duration, beans earned, peak concurrent viewers and a distinct gifter count. Its own history is instructive: it previously overstated host earnings twenty-fold by passing beans in a field named for diamonds, and reported zero viewers and zero gifters to every host.

15. Party Rooms

IMPLEMENTED, and the most carefully server-owned feature in the product. Seventeen callables.

15.1 What a room is

Multi-seat audio or video rooms with their own seats, seat requests, seat invites, listeners, messages and gift ticker.

Creation options: **mode** (audio or video), a room name, a **seat count** from an enumerated list, **seat mode** (anyone may sit, or ask first), **visibility** (public, or private with a four-digit password), and a welcome message that **rejects digits** to stop phone numbers being posted.

The seat count is enumerated rather than free, and the reason is stated: a video seat is a decoded video stream, so the choice is a device budget, and the failure mode is a room that takes the phone down.

15.2 Why seats are callables

Two reasons recorded in the code. **Contention** — one winner per transaction. And **the media token**: the token function issues a publisher slot **only to someone currently sitting on a seat**, the owner included. Without that, *"leave the seat would be a UI state a patched client could ignore"* while still broadcasting.

The token function also **decides the media identity itself**, overwriting whatever the client sent, because a client that asked for a token on somebody else's seat index would otherwise appear in their tile.

Private rooms additionally require a listener presence row, which the join call writes only after checking the password — otherwise one could skip the join, ask for a token directly, and be in the room.

The security rules match: seats cannot be created or deleted by any client, and a seat update is restricted to your own self-mute and camera flags.

15.3 Seat state — four fields, deliberately not collapsed

A moderator's mute, the occupant's own self-mute, a camera flag, and beans received. The two mutes are separate because collapsing them would let a muted guest unmute themselves. **Unmute authority is hierarchical**: an owner's mute can only be lifted by the owner.

15.4 Admin powers

Power	Owner	Admin
Rename, reshape, visibility, password, welcome message	Yes	No
Appoint or dismiss admins	Yes	No
Approve or refuse seat requests; invite; lock a seat	Yes	Yes
Mute a member; remove from a seat; kick a member	Yes	Yes

Power	Owner	Admin
Mute or remove the owner or another admin	Yes	No
Lift an owner’s mute	Yes	No
Close the room	Yes	Only once empty

A maximum of **eight admins**, for two stated reasons: an admin who could appoint admins is an owner, and the admin list is read by every permission check in the file.

15.5 Gifts in a party

The recipient is a seat holder the sender picked — never the room owner. The reason: in a party the owner is often not the person being listened to, and *“paying the room owner for someone else’s performance is the wrong money movement.”*

A long-running defect is documented here: the gift settlement was hardcoded to the live-room collection for every context, so a party room’s mirror block was skipped entirely. The money moved and **the room showed nothing at all** — no animation, no per-seat leaderboard, no room total. The security rules concede the same: *“so nothing landed here at all and every party gift appeared to vanish.”*

A non-host seat holder earns beans but gets **no daily statistics and no level progression**, because those are gated on the host flag.

15.6 Cleanup

Two fuses, deliberately different. A room emptied but never closed by its last client is swept after **2 seconds**; a room **never** occupied is swept after **10 minutes**. The 2-second fuse is really a client-closes-the-door mechanism: anyone may close an empty room, because the fastest useful cron is once a minute and *“An abandoned room lingering in the list for up to sixty seconds is exactly the ghost-room defect this was supposed to fix.”*

Closing deletes every seat, clears the in-party flags on each seat holder, and writes the room inactive — all in one batch. Listener documents are **left alone deliberately**: a room can hold more listeners than one batch allows, nothing reads a closed room’s listeners, and the presence flag is the part that would otherwise strand someone at the top of the discovery feed.

A wrong private password **increments an attempt counter and still throws**, deliberately not a lockout, because *“locking a room because a stranger guessed at it would hand any passer-by the ability to shut it.”*

16. Events

NOT FOUND as a working feature. Three fragments exist and nothing joins them.

Fragment	State
A `chance` mechanic kind called `event`, documented as *“a limited-time mechanic”*	Declared. **Nothing in the repository creates one.**
An `events` feature flag in App Control, **default enabled**	**Gates nothing.** There is no events collection, router, service or screen.
A deleted raffle-entry design	`levelEntries` carried entries per hour for hosts, entries per gift for users, and a minimum gift value for an entry. It is explicitly removed from configuration management and **deleted on every save**.

Leaderboards do exist, in two unrelated forms, and **neither awards a prize**: reporting leaderboards sent by email, and in-room live and party leaderboards. PK battles are the closest thing to a competition and have no prize mechanic.

No scheduled competitions, no campaigns with prizes, no contests, no tournaments. The word “campaign” appears only as an expense category.

17. Gifting

IMPLEMENTED. The highest-volume money path in the product.

17.1 The catalogue

A top-level collection, configured from the CRM. Writable by **Tech, App or Ops** — deliberately not all staff, because marketing and support have no gift screen. **Read is fully public and unauthenticated**, because the app shows gifts before login.

Three categories: `normal`, `exclusive`, `lucky`. The seeded catalogue:

Gift	Diamond cost	Earning rate	Category
Rose	1	50	normal
Heart	10	50	normal
Teddy Bear	50	50	normal
Crown	100	52	exclusive
Sports Car	500	54	exclusive
Rocket	1,000	55	exclusive
Lucky Box	50	**5**	lucky
Super Star	5,000	55	exclusive

The CRM only permits the earning rate to be edited freely for `exclusive`; normal and lucky are pinned to the configured value, and the form rejects a rate outside the permitted range.

> The field-name sensitivity here is called out in the code and worth repeating: reading the wrong price field would have produced a non-numeric value and thrown *“has no usable price”* on **every single gift in the app**, and checking the wrong active field would have let a gift the CRM had switched off keep selling.

17.2 The receiver’s share

Two sources, with a clear precedence. **The per-gift rate wins** when it is a number above zero and at most 100. The reason: the CRM sets that rate per gift and shows the operator the resulting host cut and ZIPPER margin on the same screen, so *“That per-gift number is the decision that was actually made.”* The category rate is the fallback for gifts seeded before the field existed.

Category	Formula	Default
`exclusive`	floor of the midpoint of a configured min and max	**52%**
`lucky`	the configured lucky percentage	**50%** in code
`normal` and anything else	the configured normal percentage	**50%**

Exclusive uses the midpoint, not a roll. Stated inline: *“a per-transaction random share would make a host’s earnings unpredictable for no benefit to anyone”*, and the lucky mechanic already exists for gifts meant to be a gamble.

> **Contradiction found.** The code's fallback for the lucky percentage is **50**, while the CRM's own form default and the seeded Lucky Box are **5**. In practice the per-gift rate pre-empts it for the seeded gift. But a hand-seeded lucky gift with no per-gift rate and no configured lucky percentage would pay the receiver **50% instead of 5%** — a tenfold overpay on the category the CRM describes as *"rest goes to prize pool."*

17.3 The send sequence

Everything below happens inside **one Firestore transaction**: it commits together or not at all.

```
Validate: signed in, gift id, receiver, NOT SELF, quantity 1..9999,
          context in {live, call, dm, party}, idempotency key <= 128 chars
    |
    v
Seven reads in parallel: claim, sender, receiver, gift, economy config, room, seat
    |
    +--> claim exists?  --> return the original result, charge nothing
    |
    v
Sender exists and is NOT BANNED; receiver exists; gift exists and is active
    |
    v
Price read FROM THE SERVER; total = unit x quantity; balance checked
    |
    v
beans = FLOOR(total x pct / 100)      "never credit a bean that was not paid for"
    |
    v
Sender: diamonds down, lifetime spend up
Receiver: beans, lifetime earned, gift earnings, gifts received
Weekly earnings bucket; if a host, daily stats and the level ladder
    |
    v
Eleven more records: private transaction, two showcases, the pair counter,
top fans, two ledger rows, the CRM reporting row, the room ticker,
room totals, the seat total, platform revenue
    |
    v
The idempotency claim, written last
```

Self-gifting is refused because it *"would let a user cycle diamonds into beans at the receiver percentage."*

17.4 Worked examples

Gift	Cost	Rate	Beans credited	Platform keeps
Rocket ×1	1,000	55%	550	450
Rocket ×77	77,000	55%	42,350	34,650
Lucky Box ×1	50	5%	**2** (floor of 2.5)	**48 of 50 — 96%**
Rose ×1	1	50%	**0** (floor of 0.5)	1
Rose ×10	10	50%	**5**	5

The floor is per transaction, so batching materially changes a host's income on cheap gifts. A Rose sent ten at a time yields five beans; the same ten sent one at a time yield **nothing at all**.

17.5 Everything the send writes

Record	Visibility	Notes
Private gift transaction	The two parties, finance, admin	Closed to all client writes
Gift showcase, sender and receiver	**Public, anonymous**	Counts only — no uid, no timestamp
Top fans, on the receiver	**Public, named**	How much, **never which gift**
Gift pair counter	Only the two parties	One document regardless of who sends first
Diamond and bean transaction rows	The owner	Carry a **running balance** and the related uid
The CRM reporting row and platform revenue	Staff	**Every gift figure in the weekly and monthly reports is built from these two**, not from the private transaction
Room ticker, room totals, seat total	In-room	The ticker drives the animation
Weekly earnings, daily stats	The host	Daily stats only if the receiver is a host

The ledger rows are explicitly built for a chargeback file rather than a history screen: the related uid *"because a display name is not an identifier"*, and the running balance *"because without one you cannot reconstruct the timeline, only assert it."*

Read access to the reporting row is room-scoped: an in-room gift is public, anything else is private to the two parties. Until recently it was readable by any signed-in account, which meant *"any signed-in client could... reconstruct who had gifted whom, across the entire platform."*

> **Two dead subcollections.** Gifts-sent and gifts-received still carry security rules, including a create rule open to any signed-in account. **Nothing has ever written them** — the repository was repointed at the showcase — *"so both sections were empty on every profile."*

17.6 Idempotency

A claim document keyed on the sender and their key, read first inside the transaction and written last. If present, the stored result is returned and **nothing is charged**. Closed to all clients.

Why: mobile networks drop responses, and *"a gift button that has visibly done nothing gets tapped again."*

> The stated reason for writing the claim last is **not correct as written** — *"a crash mid-transaction leaves nothing claimed"* — because a Firestore transaction is atomic and write order inside it has no bearing on what survives. The behaviour is right; the rationale is not.

Key generation is the client's responsibility and **the two callers differ**. The live path mints a fresh key per tap, deliberately, so two taps a second apart are two gifts — which means the guard protects against retries of one in-flight call, not against a double tap. The default path generates one key on press and reuses it for every retry, with the instruction that *"a fresh key per retry defeats the server's replay guard and charges twice."*

18. Chance, Lucky Gifting and Player Protection

IMPLEMENTED on the server. NOT REACHABLE from the app.

This is the largest undocumented subsystem in ZIPPER: roughly **4,700 lines of service code and 2,200 lines of player protection**, plus six CRM screens and five dedicated database collections. **The Android app does not reference it.** One unrelated comment is the only match in the entire Kotlin codebase.

18.1 What it is

A control plane for games of chance — lucky gifts, games, and limited-time mechanics — built to a standard that implies a regulated product.

Three entity types:

- A **mechanic** is the thing that exists. Three kinds: `lucky_gift`, `game` (*"Aviator and anything like it"*), `event`.
- A **configuration version** is **immutable**: an outcome table, stake denominations, a maximum single win, enabled countries and an RTP band. A change is a **new version**, raised as a card on the money board, verified by a second person, and **signed by an owner**.
- A **round** moves through `committed` → `playing` → `recorded` → `settled`, or `void`. *"A crash leaves it at `playing`"* — stated in the code.

18.2 The draw is provably fair

Commit-reveal with HMAC-SHA256. The seed hash is published **before** the round; the seed is revealed after; the outcome is derived so that it is reproducible and independently verifiable. A design note explains why a cryptographic source is used and never a general-purpose random number generator.

18.3 What money moves

Diamonds only, through the real MongoDB ledger — the one subsystem on the Firestore side of the product whose value movements *do* reach the business ledger. The stake and the payout are separate movements, each with its own idempotency key derived from the round.

Units are precise: probabilities in **parts per million**, multipliers in **basis points**.

18.4 Three payout ceilings and a reserve

A per-round ceiling **caps silently**. Three daily budgets and a reserve behave differently: they **trim the payout, withhold the shortfall, and automatically suspend the mechanic**.

Control	Scope
Maximum payout per round	One round
Maximum payout per player per day	One player
Maximum payout per mechanic per day	One mechanic
A platform-wide daily payout ceiling	Everything
A reserve, with headroom = reserve + stakes − payouts	Everything

Enforced by **atomic compare-and-increment**, not read-then-write, with a bounded retry.

18.5 Player protection

Every limit is **a whole number of diamonds** — anything else is refused. Windows are daily or weekly, cut in the business timezone rather than UTC, with the reasoning written out.

Three sources of a limit — the house, staff, and the player themselves — and **the lowest positive limit wins**. The code states it as a rule: *"The lowest positive limit wins, and that is the whole rule."* A staff-imposed limit cannot be loosened by the player.

Block	
`self_excluded`	Minimum **180 days**; a 72-hour reflection period on a lift request
`cooling_off`	Minimum 24 hours, maximum 90 days
`staff_suspension`	Imposed on review
`daily_loss_limit` / `weekly_loss_limit`	
`protection_unavailable`	**Fails closed** — if protection cannot be consulted, play is blocked

A limit **increase** carries a 24-hour delay. Session reminders fire on an interval. Three harm patterns are auto-detected: escalating stakes after losses, a sustained session, and rapid repeat play.

> **Two configuration facts worth surfacing.** The platform-wide daily payout ceiling defaults to **zero**. And both the default daily and weekly loss limits default to **zero, meaning no house limit is set** — the lowest-positive-limit rule means a zero is treated as "not set", so protection would rest entirely on staff-imposed and player-chosen limits.

18.6 A third-party actor that can move money

An outside game operator can **settle a round**. The request is signed with HMAC-SHA256 over a canonical string with a five-minute freshness window and keys from the environment. The payout is **recomputed on ZIPPER's side and the settlement refused if the provider's figure disagrees.** An eight-item integration contract is written into the code, including the requirement that the provider's settlement reference be stable across retries *"so their retry is our idempotency key rather than our second payout."*

18.7 Who operates it in the CRM

Tier	Who	What
Read	**All thirteen internal roles**, including the observer	Every screen but the money one
Propose	Any team admin	Preview a table, create a mechanic, propose a configuration, suspend a mechanic, review a protection flag, impose a limit
Activate / resume / withdraw	**The owner alone**	Because a mechanic that stopped by paying past a signed limit is a decision about money
Freeze a player	Owner or **Ops admin**	Two guards, and the narrow one wins
The money screens	Owner or **Finance admin**	Gated **narrower than the API**, on purpose

Self-exclusions and cooling-off periods are **rendered read-only**, because no route exists that cancels or shortens one.

18.8 Status: LIVE or NOT LIVE?

UNKNOWN — this is the single most important business question in this document.

The evidence, and nothing beyond it:

For “not live”	Against
No Android code path reaches it	Four alert detectors watch it in production
The spec quoted in the code reads *“Until this exists, no chance mechanic should be live”*, which is a prerequisite, not a launch	A lucky gift category exists in the live gift catalogue and is sold today
The platform daily payout ceiling defaults to zero	Six CRM screens are built and navigable
Both default loss limits are zero	Stakes and payouts are wired into the real ledger
	A `lucky_gifts` feature flag exists in App Control, default enabled

The lucky gift in the catalogue is settled by the ordinary gifting path, not by this subsystem. Whether the two are intended to be joined is not stated anywhere.

19. The Two Financial Systems

This is the central finding of the financial audit. It is stated before any individual money flow because every one of them is shaped by it.

19.1 The two systems

	MongoDB	Firestore
What it holds	The business ledger, current balances, approval cards, the audit log, staff, partners	The balances the app reads, and per-user transaction subcollections
Who writes it	**One function**, in the FastAPI backend	The Android app, the CRM browser, and 21 Cloud Functions
Entry shape	Transaction id, holder, currency, amount, action, **balance before**, **balance after**, reason, idempotency key, metadata	A row with a type, an amount, a reason and, on the better ones, a running balance
Atomic with its balance	**Yes** — one transaction	Yes, within Firestore
Idempotent	**Yes** — a unique index on the key	Only where a claim collection exists
Audited	Yes — an append-only log, with a mandatory reason	No actor, no reason, no before/after

19.2 They never meet

No Cloud Function writes the MongoDB ledger. It cannot: the ledger is written by one Python function, and the Cloud Functions are TypeScript running beside Firestore. There is no bridge.

This is not an inference. The code says it twice, in its own words:

> *"The app spends diamonds through Cloud Functions that Mongo never hears about, so [the post-movement balance] is only ever a lower bound on the truth, and writing it over the live balance refunded every gift and call charge since the CRM last touched this holder."*

> *"Nothing has ever compared the two. That was already true of the recharge flow before any of this, and the Cloud Functions widened it: every gift and every minute of call time now lands in Firestore and nowhere else."*

19.3 What that means in practice

```
Gift sent, call charged, beans converted, reward claimed, cosmetic bought
     |
     v
Firestore: balances move, per-user transaction row written
     |
     X   <-- nothing crosses here
     |
MongoDB ledger: no entry
```

- **Every gift and every minute of every call is invisible to the business ledger.**

- The commission engine therefore bills on something the ledger *can* see — **beans cashed out** on withdrawal — rather than beans earned. §20.3 explains why that follows.
- A nightly reconciliation job compares the two stores and **corrects nothing**. It alerts.

> **GAP-F001 — There is no single financial source of truth.** The business rule that every balance mutation produces an auditable ledger entry is **true for the backend and false for everything the app does.**

19.4 Balance mutations with no transaction row at all

Distinct from the above: thirteen places move a balance or a counter without writing any per-movement record in either store. The most consequential:

Site	What moves	Finding
Recharge approval	**A seller's coin balance**, by direct set	A buyer transaction row is written. **Nothing at all is written on the seller side.** No coin-transaction collection exists anywhere in the product. A seller's inventory is decremented with no per-movement record.
Call ticks	Payer diamonds, earner beans	Charged every ten seconds with no row; a **rollup row** is written when the call ends. A call that never reaches its end moves money with no record.
Weekly earnings accrual	The week's bean bucket	Commits inside the caller's own transaction, which is the point, but the bucket itself is unlogged.
Bean conversion	A week's unreleased bucket	User-level rows exist; the per-week redistribution has none.
Video card minting	**A consumable asset**	Created silently by the daily reward. The adjacent row records the diamonds only.
Withdrawal status writes	Nothing — status only	Three browser writes record `processedBy: 'Admin'` as a **literal string**. No actor, no audit.

The Android app is clean. No Kotlin code writes any balance field. Every reference is a read or a callable result. The only client-side increments are non-money: unread counts, viewer and like counts, and PK scores.

19.5 Idempotency coverage

Guarded	Mechanism
The MongoDB ledger	A **unique index** on the idempotency key
Gifts	A claim document per sender and key
Calls	A claim document, plus a server-side tick counter
Bean conversion, cosmetic purchase	A claim document
Daily reward	The date itself is the key
Chance stakes and payouts	A key derived from the round; the external provider reuses it
Commission accrual	Keyed on **agency + host + period**
Email	A unique dedupe key per recipient

> **GAP-F002 — The recharge approval has no idempotency key.** It is the highest-value unguarded path. Its only guard is the status transition itself, and that guard is **deliberately left loose enough to re-fire**: on failure the request is sent back to pending so the next approval retries. The comment explains why the retry is needed — the buyer has already paid cash outside the system — but does not pair it with a dedupe key. **Two approvals of the same request transfer coins and credit diamonds twice.**

20. The Commission Engine

IMPLEMENTED. Treated here as the current, correct business logic.

> **A documentation note, with no judgement attached.** Two comments in the codebase still describe the rule as a flat rate per band. The executable code, the rule string it publishes to partners, and its test suite are **progressive**. The tests record the change explicitly: *"It was flat by band ... and the owner confirmed against a worked example that it should be progressive."* A separate business description using the word "ratio" also exists; reconciling the three is a Phase 2 matter and is **not** done here.

20.1 The slab tables

Live tables are read from Firestore configuration. The documented fallbacks, dated from a backup, are numerically identical to the CRM's own defaults.

Main agency

Band	Billing from	Billing to	Rate
1	₹25,001	₹75,000	5%
2	₹75,001	₹1,50,000	7.5%
3	₹1,50,001	₹3,00,000	10%
4	₹3,00,001	₹5,00,000	14%
5	₹5,00,001	₹8,00,000	16%
6	₹8,00,001	₹12,00,000	18%
7	₹12,00,001	₹18,00,000	21%
8	₹18,00,001	₹25,00,000	23%
9	₹25,00,001	no upper limit	25%

Floor: ₹25,001. Below it, commission is a real zero — not an error.

Sub-agency

Band	Billing from	Billing to	Rate
1	₹26,001	₹75,000	5%
2	₹75,001	₹1,50,000	6%
3	₹1,50,001	₹3,00,000	7%
4	₹3,00,001	₹5,00,000	8%
5	₹5,00,001	₹8,00,000	9%
6	₹8,00,001	₹12,00,000	10%
7	₹12,00,001	₹18,00,000	11%
8	₹18,00,001	₹25,00,000	12%
9	₹25,00,001	no upper limit	12.5%

Floor: ₹26,001 — a thousand rupees higher. At the top band a sub-agency earns **half** what a main agency earns on the same billing.

Two details about the tables themselves:

- **Band 9 is open-ended** regardless of its declared upper bound. The declared figure is retained for display only.

- **A band's effective upper bound is the next band's lower bound minus one paisa**, not its own declared figure. So band 1 effectively runs to ₹75,000.99, and the 99 paise between ₹75,000.00 and ₹75,001.00 stay in the band below.

20.2 The arithmetic

It is progressive. Each slice of the billing is charged at the rate of the band it falls in. The published rule reads: *"Progressive by band, like income tax… No single rate is ever applied to the whole amount."*

For each band reached, the slice is the billing capped at that band's ceiling, less the band's lower edge. Each slice is multiplied by its own rate in basis points, the products are summed, and **the rounding happens once, on the total** — half up, to the nearest paisa, **ties to the agency**.

Worked example — main agency, ₹10,00,000 of billing

Band	Rate	Slice	Contribution
1	5%	₹50,000.00	
2	7.5%	₹75,000.00	
3	10%	₹1,50,000.00	
4	14%	₹2,00,000.00	
5	16%	₹3,00,000.00	
6	18%	₹1,99,999.01	

Band 7 begins above the billing, so the loop stops.

Commission = ₹1,35,124.82. The effective rate is **13.51%** — well below band 6's headline 18%. Six bands were crossed, and ₹25,000.99 was below the floor and untaxed.

The flat-by-band figure would have been ₹1,80,000 — a difference of **₹44,875.18** on one agency for one period.

A second owner-confirmed example: ₹1,00,00,000 of billing yields **₹23,33,124.75**, crossing nine bands.

Exactly at a band boundary

At a band's own lower bound, that band contributes **exactly one paisa**. The consequences are asserted in the tests:

Billing	Commission
₹25,001 exactly (the main floor)	**₹0** — matched, band 1 named, figure zero
₹26,001	₹50.00
Sub-agency at ₹26,001 exactly	**₹0**
Sub-agency at ₹27,001	₹50.00

There is no cliff at a boundary. The step from ₹75,000 to ₹75,001 is under a rupee, and monotonicity is swept across a wide range with zero decreases.

20.3 What "billing" means

The published definition: *"An agency's billing for a period is the total its hosts **cashed out** in that period: bean debits on a host's balance whose action is a payout and whose metadata names the agency, valued at the bean-to-rupee rate in force."*

It is beans CASHED OUT, not beans EARNED. This follows directly from §19: beans earned are a Firestore movement the ledger never sees; beans cashed out are written to the ledger by the withdrawal execution, and the code notes that this shape is *"exactly the shape the commission engine aggregates on."*

The engine also reports beans **earned but not cashed out**, as information, and never adds it to billing. When hosts earned but cashed nothing out, a warning explains that the period genuinely bills nothing and *"those beans will bill in the period they are cashed out."*

The agency total is converted from the agency's **total** beans once, so it can never disagree with the sum of the per-host figures by a rounding step. Per-host rupee figures are explicitly for display only.

> Two of the two recognised payout actions exist in the code; **only one is ever written** by any service. The other has no writer anywhere.

20.4 Apportionment across hosts

One agency-level figure is split across hosts by **largest remainder**, weighted by beans cashed out. Each share is the exact proportion floored; the leftover paise go one each to the largest remainders, with a deterministic three-level tie-break. **The parts sum to the total exactly.**

Each host line carries a note stating that *"the band was chosen by the agency's total, and this is that commission's share, not a separate calculation."*

20.5 Periods, accrual and payment

Valid period labels: an ISO week or a calendar month. Anything else requires an explicit window. Bounds are **half-open** — start inclusive, end exclusive — *"so two consecutive periods can never both claim the same cash-out."* Periods are capped at 400 days. The window is computed in **UTC**.

Accrual is a positive ledger movement on the agency's commission balance, in paise, keyed idempotently on **agency, host and period**. The metadata carries the **entire derivation** — band, rate, floor, rule, rounding, the explanation, the host's beans, the share and its note, the table source, and whether the figure was provisional.

It is driven automatically by a Cloud Function at **Sunday 18:29 UTC**, which writes the weekly report first and then accrues for **every approved and active agency**, not only those with eligible hosts.

Four rules the unattended path adds

Rule	Behaviour
Preview first, then pass the figure back as an expected amount	A mismatch refuses and credits nothing
Provisional rates refuse outright	Returns a conflict; nothing is credited
A sanity bound — ₹12,50,000	Refuses rather than truncating. Justified as 25% of the top band
Zero is an outcome, not an error	Reported as "nothing to accrue"

Refusals are audited under their own action, so a declined accrual is distinguishable from one that never ran.

Paying it out is a separate card: raised against the outstanding balance, refused if it would overpay, and **executed by an owner** with a mandatory payment reference.

20.6 Parent and sub-agency money, as implemented

- A sub-agency's commission is computed on **its own** billing, from **its own** slab table.
- A parent's billing includes only hosts naming the parent. **There is no roll-up.**
- **The parent earns nothing from the sub's production.** No override, no parent share, no spread. Searched for and absent.
- The parent is a **payment conduit**: ZIPPER pays the parent, and the parent then pays the sub (§9.5).

20.7 Adjustments and reversals

NOT FOUND.

An exhaustive search for a reversal, adjustment or clawback path returns nothing. The supporting evidence is strong:

- The commission audit vocabulary is exactly three actions: accrue, accrue-refused, pay. **There is no reversal action.**
- The ledger rejects a zero movement and rejects any movement taking a balance below zero unless explicitly permitted, and that permission is used **once** in the entire services layer — for a host bean withdrawal, not for commission.
- The audit module has no update or delete path at all: *"Nothing in this module updates or deletes. There is deliberately no such function — not even for owners."*
- Accrual only ever applies **positive** movements.
- The idempotency key makes re-running a period a no-op per host, so there is **no re-accrue-at-a-corrected-figure path either.**

20.8 Provisional figures

A "provisional" figure is one computed from a documented fallback rather than from live configuration. The fallback table is used in exactly three cases: the configuration read threw, the configuration block is absent, or the specific key is missing. **A present-but-malformed table does not fall back — it raises**, with a distinct status code.

The fallback note reads *"PROVISIONAL — … must not be paid without confirming the live configuration."* Being provisional appends a warning, marks the computation, and **refuses an unattended accrual outright**.

The malformed-table check is strict: it refuses a non-list, an empty list, a non-mapping entry, missing bounds or rate, a fractional or negative bound, a rate outside 0–100 or finer than a whole basis point, an inverted band, overlapping bands, and a real gap between bands. Tables merely out of order are **sorted, not refused**.

21. Host Earnings and the Weekly Release

IMPLEMENTED.

21.1 The cycle

```
A gift or a call credits beans
    |  and, in the same transaction, a WEEK BUCKET for the current IST week
    v
Week N closes Sunday 23:59 IST
    |
    v
Monday 00:30 IST — the release job runs, 31 minutes later
    |  queries unreleased weeks at or before the cutoff, 500 at a time
    |  reads the bean rate ONCE so every statement in the batch settles alike
    v
Two gates, inside the transaction:
    |  isVerifiedHost must be true     --> else SKIPPED, not lost
    |  payoutHold must not be true     --> else SKIPPED, not lost
    v
beansAvailable increases; the week is marked released; a transaction row
and a weekly statement are written
    |
    v
The host raises a withdrawal; Finance pays by bank transfer;
an owner executes a card; a ledger row records it
```

The hold is one week. The code describes it precisely: *"A Monday gift waits about thirteen days, a Sunday one about seven."*

21.2 The two gates, and why a skip is not a loss

A blocked week **rolls the transaction back**, so it stays unreleased and **the next run picks it up once the block clears**. Outside the transaction, the week is stamped with a reason and a statement is written marked *held*, with the reason and the pending amount — deliberately outside the transaction, because otherwise the held weeks would be invisible to the host.

The verification gate's reasoning is recorded: *"Sending money to an identity nobody has confirmed is the exposure India's AML rules exist for."*

21.3 Converted beans leave the payout pool

Converting beans to diamonds draws down **released** beans first, then the oldest unreleased week. Otherwise a host could be *"paid once in diamonds and again in rupees for the same earning."* The weekly statement records the converted amount so the shortfall is explained rather than mysterious.

21.4 Eligibility rules, with their values

Rule	Value
Weekly minimum payout	From configuration; **fallback ₹2,000**
A second bean-count condition	Supported, applied **only if configured**
…its current state	**NOT CONFIGURED.** The code says: *"The owner has described a second rule — 'one lakh beans in a week' — and it exists nowhere in this system"*, and names the key that would enable it
Verification	Unverified → the week is skipped, not lost
Payout hold	Held → skipped, not lost (but see GAP-005: nothing can set it)
Release timing	One week in arrears
Withdrawal ceiling	Cannot exceed released beans; re-checked inside a transaction at execution
App-side minimum	**100,000 beans = ₹1,000**
Eligibility window vs commission window	Eligibility uses a **rolling 7 days** ending when the job fired; commission uses the **ISO week** from the label. *"The two do not overlap and are not meant to"*

A host below the line always receives the figure and **the exact shortfall** — *"₹X earned against a ₹Y weekly minimum — ₹Z short"* — never a bare refusal.

21.5 The weekly payout report

Written by the Sunday job and carrying one row per agency: the agency, its type, whether it is foreign, host and eligible-host counts, total beans, the rupee payout total, and the commission outcome.

It deliberately carries **no local commission figure, no effective rate and no grand total** — those come from the accrual, so the report and the credit cannot drift apart.

Report statuses: generated, submitted, approved, rejected. Approval writes **approval markers only** — *"No balance is written here… beans stay exactly as the host earned them until the payout is executed on the money board."*

> Two different collections share the name `hostPayouts`: a subcollection of the weekly report, and a top-level one. The code names this collision explicitly, and it is why host statements were put somewhere else.

22. Salary

PARTIALLY IMPLEMENTED. A Firestore-only workflow with no ledger involvement and no audit.

22.1 The flow

```
THE AGENCY raises a request from its own dashboard
    |  host, period start, period end, beans earned, rupee amount, notes
    |  born "pending"
    v
FINANCE approves          --> status approved
    v
FINANCE marks it paid     --> status paid, UTR REQUIRED
    v
THE AGENCY marks it distributed  --> status distributed
                                     (the agency confirming it passed the money on)
```

A rejection path exists from pending.

22.2 What is and is not recorded

Aspect	State
Who raises it	The agency
The figures	**Both typed in by the agency.** Nothing derives them from the ledger
Validation	Client-side only: a host, both dates, both numbers
Who approves and pays	Finance admin or the owner. The rule was deliberately narrowed from "any admin" because *"marketing_admin and support_admin could mark a host payout approved and paid from a browser"*
Payment reference	**Mandatory** to mark paid
Period	**Free-text strings** chosen by the agency. Not an ISO week, not a month, not resolved through the period function
Denomination	**Rupees**, as a plain number — not minor units
Does it touch beans?	**No.** Every write is to the request document only. No bean field, no transaction row, no ledger movement, **and no Cloud Function trigger at all**
Backend functions	**None.** The card type exists; nothing in the backend raises or executes one

> **A naming hazard the reports call out:** the `staff_salaries` expense category is **not** the salary card type, which pays *hosts* and is already counted under host payouts. *"The two have similar names and are different money."*

22.3 Does salary count toward commission billing?

No. Evidenced in both directions:

- Billing is filtered to two payout actions. A salary payment would have to carry one of them.

- **The salary flow writes no ledger row at all.** The aggregation reads the MongoDB ledger; a Firestore salary document is invisible to it — no holder, no currency, no agency metadata, no action.
- The reports treat them as different money: an executed salary **card** is counted under host-payout expenditure, a cash-basis line, while commission billing is a ledger-movement basis.

> **GAP-007 — The salary request and the salary card are unconnected.** The reports count executed salary cards. Nothing in the backend raises one, and nothing reads the Firestore salary queue to produce one. Whether a host salary request ever becomes an executed card is **not determined from the implementation**.

> **GAP-008 — No actor and no reason on any salary transition.** Approve writes a status and a timestamp; mark-paid writes a status, a timestamp and the UTR; reject writes a status alone. **A host's salary is approved and paid with no record anywhere of who decided it or why.** By contrast, a money card rejection without a reason is refused outright.

23. Withdrawals

IMPLEMENTED on the authoritative path. The Firestore queue has no producer.

23.1 Two parallel mechanisms

A. The money-board host withdrawal — authoritative

This is where the money actually moves.

```
Request raised  (reason required, beans positive, host named)
    |  the host's wallet is READ; an unreadable wallet refuses outright,
    |  because "there is no figure to check this withdrawal against"
    |  ceiling: cannot exceed released beans
    |
    |  two WARNINGS, not blocks:
    |    no agency on the record -> "this payout will bill no agency
    |                                and no commission will be earned on it"
    |    not a verified host     -> the weekly release will not release another week
    v
A card: Finance team, the host as the subject
    v
Verified, then SIGNED BY AN OWNER
    v
Executed, with a MANDATORY payment reference
    |
    |  STEP 1, Firestore first: debit released beans under a transaction
    |          that re-reads the balance. A shortfall fails the card
    |          with both figures named.
    |
    |  STEP 2, the ledger row: the movement the commission engine bills on
    |
    |  a step-2 failure leaves the card failed, flagged as needing
    |  manual correction, with the Firestore debit already applied
```

The ordering is justified inline: Mongo first would leave *“a ledger saying the host was paid while the beans are still in their wallet.”*

B. The Firestore withdrawal queue

Four statuses — pending, approved, rejected, paid. A Finance screen reads it, with approve, reject-with-a-reason and mark-paid-with-proof actions. Payment modes offered: NEFT, IMPS, UPI, Binance, USDT TRC20, USDT ERC20, Bank Transfer.

> **GAP-F005 — This queue has no producer.** The security rules permit a host to file one. **No code in any codebase files one** — not the app, not the CRM, not a Cloud Function. The Finance withdrawals screen reads a collection nothing populates.

> **GAP-F004 — Three browser writes with no actor and no audit.** Approve, reject and mark-paid are plain browser updates that record `processedBy` as the **literal string `'Admin'`** — not the acting person's identity. There is no audit entry. The security rule asserts that *"approving records a decision, and the money itself moves as a signed card"*, and the card path does audit — but **nothing links the two**, so a request marked paid with no card behind it is indistinguishable from one with.

23.2 Limits, fees and gates

Minimum (app-side)	**100,000 beans = ₹1,000**
Weekly eligibility minimum	Configured; fallback ₹2,000
Maximum	**None**, beyond the released balance
Fees deducted	**None anywhere in the implementation.** Two percentages exist in configuration — a Binance fee and a government tax — and the reports state explicitly that both are **quoting rates that were never a booked cost**; the booked cost is whatever left the business on an expense card
KYC	No field or check named KYC in this flow. The equivalent gate is **verification**, which blocks the *release* into withdrawable beans and therefore caps what can be withdrawn
Payment proof	A mandatory reference on the card, stored on the ledger movement; plus UTR, transaction reference, a proof image, a date and a mode on the Firestore document

24. Recharge and Payments

PARTIALLY IMPLEMENTED. There is no payment gateway.

24.1 How money enters

Two paths, both out of band.

A. A user buys diamonds from a seller

```
Diamond Shop: "Zipper sets the pack price. You pay a verified reseller
               directly and they may add their own margin."
    |
    v
The user chooses a seller from the verified directory;
a pre-written message is handed to them
    |
    v
The user pays the seller DIRECTLY — cash, UPI, bank transfer —
and sends a screenshot
    |
    v
A recharge request is filed, with the payment proof
    |
    v
THE SELLER APPROVES IT in their own app panel
    |  ONE FIELD CHANGES: status -> approved
    v
A Cloud Function fires. Inside one Firestore transaction:
    seller coinBalance DOWN, buyer diamonds UP,
    a buyer transaction row, a notification
```

B. A seller buys inventory from ZIPPER

```
The seller's app screen shows the margin in plain terms, then
ZIPPER's OWN bank details or Binance UID, read from configuration
    |  and refuses to proceed if those cannot be read:
    |  "Do not transfer to any account you may have used before"
    v
The seller transfers out of band, then submits a UTR and a screenshot
    v
A purchase request; Finance runs a LIVE DUPLICATE-UTR CHECK
    v
A card: verified, then SIGNED BY AN OWNER, then executed
    |
    v
Coins appear in the seller's inventory
```

24.2 What does not exist

> **GAP-F006 — Payment gateway integration is NOT FOUND.** There is no gateway SDK anywhere in the repository. There is no callback endpoint, no webhook handler, no order-creation flow, no payment verification step, and no refund or chargeback path. The payment mode is a **free-text field**. Gateway and Binance fees exist only as **expense categories**.
>
> One collection is read by a settlement-mismatch alert detector and would be where gateway settlements landed, but nothing writes it.
>
> Consequently, the entire failure matrix normally associated with payments — money deducted but currency not credited, currency credited but payment failed, a missing callback, a duplicated callback, a refund after the currency was spent, a chargeback after settlement — **has no implementation to audit.** Those risks are carried by the human processes around the bank account, not by code.

24.3 The recharge retry, and its hazard

The recharge trigger fires only on the transition **into** approved. On failure it writes the request **back to pending**, with the error, the failure time, an attempt count and the seller's own words — so that the next approval fires again. The reasoning is sound and recorded: the buyer has already paid cash outside the system, so the only honest outcomes are retry or refund, and the previous code offered neither.

The hazard is that `approved` **is therefore not terminal**, and there is no idempotency key (see GAP-F002). The transaction is atomic but not idempotent.

> Also noted: the approval is written **from the seller's phone** as a one-field update, and **no audit entry is produced at all** — no inventory debit action, no balance credit action, no actor, no reason. The code makes exactly this argument about host decisions elsewhere — *"a decision about whether somebody may earn on this platform, recorded somewhere no owner ever looks, is the same failure in a different collection"* — and the recharge path is the same failure, unaddressed.

25. Lending

IMPLEMENTED. The business extends coin credit to sellers on trust.

A lending record carries an amount, who lent it, notes, a status of outstanding or repaid, and the dates. **Terms are a required due date plus optional free text.**

Four acts, each a card on the money board:

Act	Card type
Lend inventory on credit	`lending_issue`
A seller repays	`lending_repay`
Recover owed coins from the seller's stock	`lending_recover`
Write off a debt	`lending_write_off`

Recovery carries a penalty, described in the code as *"a legitimate addition — a late penalty or recovery cost"*, and it may legitimately exceed the debt. The penalty accumulates on the seller's credit record.

Credit state lives on the Mongo partner record: outstanding, due date, total lent, total penalty, total written off. Standing is derived (§10.1). An overdue seller is **blocked from taking new inventory**, and the CRM surfaces the reason in the API's own words rather than as a failed request.

The deleted browser version, documented in place

Two code comments record what this replaced, and they are the clearest statement of why the card board exists:

> *"Lending is credit we extend, so it is money leaving the company on trust. It went out with no verification, no owner signature and — worse — no due date."*

> *"One click cleared a debt. No money had to arrive, no payment reference was recorded, nobody countersigned."*

Both end with the instruction **DO NOT REINTRODUCE**, and the security rules now refuse browser writes to a seller's coin balance.

26. Capital, Expenses, Liabilities and Profit

IMPLEMENTED. A three-way distinction the product owner may not have named explicitly.

26.1 Owner capital — neither revenue nor cost

Two card types: capital introduced, and drawings. The governing sentence is in the code: *"Both movements are real cash and both must be visible. **Neither is profit.**"*

A capital contribution is an owner funding the business from their own pocket. It is raised as a card on the same five-stage board, and **an owner cannot sign their own** — the reason is recorded: *"'how much did I put in' is a question three partners will one day disagree about, and the answer needs two names on it."*

Recording moves nothing. A separate **owner-only settlement** is the money actually moving. Per-owner reporting shows introduced, drawn and outstanding from **executed cards only**, and outstanding is explicitly *"a **liability**, not a pot to spend."*

> **No capital failure path.** The settlement passes success unconditionally, so a capital transfer that fails at the bank cannot be recorded as failed.

26.2 Expenses

Denominated in paise. Eight categories: staff salaries, infrastructure, marketing, gateway fees, Binance fees, taxes, refunds to users, and other.

Seven of the eight are reported; `other` is excluded, and the reason is a good one: *"an empty `other` is genuinely no information, whereas an empty `staff_salaries` means nobody entered the payroll."*

Each expense carries a monthly period label, a mandatory reason, **duplicate-invoice-reference detection**, a repeat path for recurring costs and a recurring worklist. Execution requires a payment reference.

26.3 Liabilities — three, in three places

Liability	Definition
Agency commission	Accrued less paid
Beans created	*"Future cash liability created this period"*
Owner capital outstanding	Introduced less drawn

26.4 Profit — two different formulas

Both exist, and both are used.

A. The margin report

```
net_margin = revenue − payouts − (commission accrued − commission paid)
margin_pct = net_margin / revenue × 100      (null when revenue is zero)
```

B. The profit statement — the owner's definition

```
profit = revenue − expenditure, summed over the KNOWN lines
```

Its design point is **honesty about gaps**: an unknown expenditure line is `None`, **never zero**. With anything missing, the statement is marked incomplete and the profit is explicitly described as **an upper bound**, with the caveat rendered *above* the number.

26.5 The definitions block

The reporting service carries a single canonical set of definitions, including one introduced to replace four that disagreed:

> **Payout** — *"One definition, replacing the four that disagreed."*

> **Commission** — accrued is positive movements on an agency's commission balance in the period; paid is negative movements; amounts are in paise.

27. Financial Reconciliation

PARTIALLY IMPLEMENTED — detection without correction.

A nightly job at 03:00 IST compares the previous day's Firestore diamond and bean movement against the MongoDB ledger. It **corrects nothing.** It alerts.

Its own words: *"Two stores that are never compared do not stay equal. They drift for ordinary reasons — a function that retried and wrote twice, a transaction that partly failed, a rate someone changed in the middle of a day."*

Where divergence is found, the direction is fixed by policy: *"a drift entry means Mongo is right and Firestore is stale — never the other way round."*

Sixteen alert detectors run every thirty minutes, with suppression keyed on a fingerprint so that **a condition that gets worse re-alerts rather than being suppressed**. If the suppression store is unreadable, the alert is **sent without dedupe rather than dropped**.

Detector	Severity	Told
Bank details changed	**Critical**	Owners, observer
Rate feed failed	Warning	Owners, Finance
Duplicate payment reference	**Critical**	Finance, owners
A large card awaiting signature	Warning	Owners
Execution failed	**Critical**	Finance, owners
Settlement mismatch	**Critical**	Finance, owners
Queue ageing	Warning	Owners, plus the item's own team
Staff access changed	Warning	Owners, observer
Failed logins	**Critical**	Owners
Credit overdue	Warning	Finance, owners
Email delivery failures	**Critical**	Owners, observer
Chance mechanic suspended	**Critical**	Owners, Finance
Chance reserve low	Warning	Owners, Finance
Chance RTP divergence	**Critical**	Owners, Finance
Chance player anomaly	Warning	Ops admin, owners

Alerts cannot be muted, deliberately: they pass neither a report name nor a cadence to the subscription system, and *"an alert somebody switched off is not an alert."*

> **GAP-F007 — The most critical detector cannot fire.** The bank-details-changed detector matches on two audit actions. **Nothing writes either of them.** Both are declared; the only references are reads. Meanwhile the configuration that matters — the company bank account and Binance UID that every coin seller pays into, and the economy document holding the commission slabs and the bean rate — is edited by **direct browser writes to Firestore with no audit entry at all.**
>
> The detector's own stated purpose: *"A changed beneficiary account is how a payout run is diverted, and it is cheap to verify and expensive not to."* As implemented, **redirecting the company's incoming revenue account produces no audit entry and no alert.**

27.1 Can every figure be traced? — the fifteen questions

Question	Answer
Can every rupee entering ZIPPER be traced?	**Partly.** A seller's inventory purchase is a card with a payment reference. A user's cash payment to a seller is **outside the system entirely** — the only record is a screenshot and the seller's approval.
Can every rupee leaving be traced?	**Yes** for card-executed payouts, each with a mandatory reference. **No** for the Firestore withdrawal queue, which has no actor.
Can every coin be traced from creation to consumption?	**No.** A seller's coin balance is direct-set with no per-movement record in either store (GAP-F003).
Can every wallet balance be reconciled?	**No.** Firestore and Mongo are compared nightly and not reconciled.
Can every commission be explained mathematically?	**Yes** — the full derivation is stored on the movement and shown to the partner.
Can every host earning be reproduced from transaction data?	**Yes** for the rollup rows and weekly buckets. **Per-tick charges have no individual row.**
Can every agency earning be reproduced?	**Yes** — the statement reads ledger entries rather than re-deriving from today's rates, *"which is what makes a March statement still say what it said in March."*
Can every refund be traced?	**No refund path exists.**
Can every reversal be traced?	**No reversal path exists for commission.**
Can every withdrawal be traced?	**Yes** on the card path.
Can every manual adjustment be audited?	**Yes** — a balance adjustment is a card an owner signs.
Can reports be reconciled against the ledger?	**Yes** for Mongo-sourced reports. Gift and call figures come from Firestore collections.
Can duplicate transactions be detected?	**Yes** where an idempotency key exists; **no** for recharge approval.
Can balance inconsistencies be detected?	**Yes**, nightly. Not corrected.
Can a transaction be reconstructed years later?	**Yes** on the ledger, which carries balance before and after.

28. The CRM

78 routes. **IMPLEMENTED.**

28.1 Two role vocabularies, and why it matters on every screen

Where	Role names
CRM and Firestore	`superadmin`, the twelve department roles, `agency`, `seller`
Backend and MongoDB	`owner`, `observer`, the same twelve, `agency`, `seller`

`superadmin` is the CRM's spelling of `owner`. **`observer` exists only on the API side.**

The consequence is live on every screen: a page gating on the **CRM** role grants the superadmin everything by bypass, while a page gating on the **API** role checks for `owner` instead. Several screens read both, and gate different actions on different identities.

28.2 Signing in

Three doors, plus two public application forms and an observer door, all advertised on one page.

1. **Email and password.** Firebase authenticates, then the CRM record is read **by uid, never by email**. No record means signed out with "CRM account not found"; an inactive record is refused by name.
2. **Google.** A Google identity with no CRM record is **refused, not adopted**. If the address already has a password account, the person must prove ownership with the password once, which then links Google to the same identity. An **unverified** email address blocks linking outright, because the identity provider would silently remove the password provider.
3. **The observer's one-time code** (§4.1).

A superadmin faces a further gate every login: three personal questions, set on first login, three attempts, then a forced sign-out.

A live listener keeps the role current and **force-signs-out the moment the record becomes inactive**. The CRM also exchanges its session for a separate API session; money and audit screens wait on it.

There is **no middleware**. All routing is client-side in the application shell.

One piece of care worth noting: the signed-in test is isolated in its own module with a spec, because *"a null record is not evidence of being signed out — it is also the value for 'not arrived yet' and 'the read failed.'"* Exactly one combination of facts means signed out.

28.3 The money board — five stages

Stage	What it means	Who moves it
Requested	Somebody has asked for money to move. Nothing has happened.	Anyone who may write
Under Review	Picked up and being checked.	The verifier
Awaiting Sign-off	Checked, **with a written note saying what was checked**.	The verifier; the note is mandatory
Approved	An owner has signed. **The money has still not moved.**	The owner alone
Executed	The money moved, with the ledger result recorded.	The executing service

The board states it plainly on the column headers: *“Signed — money has NOT moved”* against *“Money moved.”*

Rejected means refused before execution with nothing moved; a reason is mandatory, and only an owner may reject a card already awaiting signature. **Failed** means an owner signed and the transfer did not go through — the card is left **visibly stuck rather than silently lost**. Before that state existed, such a card simply vanished.

> **A failed card is a dead end.** No function accepts it as a starting state. It can be rejected but never retried or re-executed, which the code names as the intent.

A payment proven by a bank or gateway record is marked auto-verified and **starts at Awaiting Sign-off**, skipping review.

28.4 Twenty-two card types

Type	What it is
`balance_credit` / `balance_debit`	Add or remove diamonds or beans on a user’s account
`seller_coin_purchase`	A seller has paid for inventory; executing is when the coins appear
`lending_issue` / `lending_repay` / `lending_recover` / `lending_write_off`	The credit book
`bad_debt_write_off`	A debt judged unrecoverable
`commission_payout`	Paying an agency its earned commission
`subagency_payout`	Paying a sub-agency **through its parent** — two legs
`host_withdrawal`	A host cashing out beans
`salary_request`	A salary payment request
`weekly_payout`	The weekly payout run
`rate_change` / `pricing_change` / `fx_override`	Rates, prices, a manual currency override
`promotion`	A giveaway or discount that costs money
`refund`	Money given back
`chance_config_change`	A game’s odds table, stakes, maximum win, countries — **its own type** so that *“every rate change this quarter”* does not start returning probability tables
`operating_expense`	Money leaving to run the company; the category is data on the card
`owner_capital_in` / `owner_drawings`	**Not revenue, not a cost**

Eight of the twenty-two are readable by the partner they concern — an allowlist on purpose, so a new type is invisible to partners until added in a diff. A partner sees the reason on a rejection but not internal notes, and staff names are replaced by “Zipper”.

28.5 The three queues a person opens first

Screen	For	Scope
Waiting on you	The one page to open first. Counts three kinds of work: **Money** (cards to verify or sign), **People** (partner and host applications and changed details), **Permission** (a staff member wanting something their role does not cover). **It counts; it never decides.** A count that could not be read renders as a dash, never zero.	All staff, plus the observer

Screen	For	Scope
Money Board	The whole pipeline. Owners and the observer see every team; a team admin sees their team; a member sees their own and their team's; a partner sees only their own.	All staff, plus the observer read-only
Approval Queue	The superadmin granting or refusing requests from staff whose own role does not permit an action. **Twenty-eight request types.**	Superadmin only

28.6 Finance screens

Every finance page is gated to **superadmin and finance admin**, plus the observer. `finance_member` is deliberately excluded, and the reason is recorded: the API admits only a team admin or a read-all role, so listing members *"put them on screens where every figure came back 403 — which reads as the CRM being broken rather than as a boundary."*

Screen	The question it answers
Overview	What came in this week, what we owe now, what is waiting, what is overdue or stuck
Cash	Pick a week, get the receipts. **Protects the distinction that cash received is not consumption revenue**
Liabilities	What is owed right now — a point in time, so no date range
Payouts	Which weekly reports await a decision, and which cards were **signed and never executed**
Withdrawals	Which cash-outs are pending, and which are **approved but not paid**
Profit	Margin, bean-payout ratio, revenue over time, reseller versus direct
Costs	What it costs to run the company — *"the second half of profit, which had nowhere to be entered"*
Owner Capital	What each owner put in, took back, and is still owed
Weekly Reports	The payout run, one row per agency
Credit Book	What is out, what is late, per seller
Seller Sales	*"The screen that makes inventory come down"*
Coin Purchases	Where money genuinely enters the business
Economy / Config / Currency	Rates and slabs; a superadmin saves directly, everyone else **submits for approval** and the button changes its own label

Three conventions these screens share, each stated repeatedly:

- **An empty result renders as "no data for this period" and never as ₹0**, *"because a confident zero is worse than an error, since nobody double-checks a zero."*
- **Provisional** means a rate was missing and a documented fallback was used, said **above** the number.
- **A read that failed is never rendered as zero.**

28.7 Operations, support and oversight

Screen	Notable behaviour
Report queue	Every decision **calls the API, which carries it out**. Previously the queue recorded decisions that were never enforced and the reporter was told it had been dealt with
Device bans	Searchable by full device id. Distinguishes a **cascade** ban, from banning an account and lifted only when every source account is unbanned, from a **manual** one
Appeals	The only queue whose top row is a person currently locked out. **Oldest first, not a preference** — *"waiting costs the appellant every day."* Overturning is not one thing, so **two lists** are shown every time: what was reversed, and what remains outstanding
Live monitor	Nothing writes to Firestore. Three things it refuses to hide: the polling interval is visible and pausable; ending a room only sets a flag; and **muting is a flag the app must honour** — the API is not in the media path and cannot silence anybody
Alerts	Three questions kept apart: **Open now**, **Raised** (the suppression record that answers *"I was not told"*), and **Watched** (the catalogue of detectors and who is told). Releasing a suppressed alert is **owners only**
Throughput	Cards per stage, **medians not averages**, the oldest open item named. A stage nobody completed shows a dash, not zero, *"because a zero would read as instant"*

Screen	Notable behaviour
Jobs board	Five outcomes never collapsed: **ok**, **degraded** (*"ran, delivered what it could, and something in it did not"*), **failed**, **never** (the loudest, because it is the failure nobody notices), **unknown**. `stalled` is independent. The observer reads it in full and is offered **no re-run**
Email delivery	Four outcome groups kept apart, with **`queued` counted as a failure** because a stuck row means the outcome is genuinely unknown. Failures first, *"because a bounced payout receipt is money the recipient does not know about"*

28.8 Tech and the kill switch

App Control holds maintenance mode with a customer-facing message, force-update with a minimum version, and **ten feature flags**: live streaming, voice calls, gift sending, lucky gifts, leaderboard, coin shop, bean withdrawal, referrals, **events**, agency portal.

The important rule: **a superadmin's save writes immediately; a tech admin's or member's save becomes an approval request** — *"Change submitted — awaiting superadmin approval."* Flags new to the code are merged in as **enabled** on read.

> The kill switch is **not in the sidebar.** Its only entry point is a card on the Tech dashboard.

28.9 Configuration screens

All seven App Configuration pages are **owner-only**. The premise: all hardcoded app data was migrated to Firestore so it can be managed without shipping a new build.

Page	What it configures
Bag items	The minimum user and host level to unlock each cosmetic
Packages	User recharge packs, seller wholesale packs, coin packages, local prices per country
Daily rewards	The seven-day streak: diamonds and video cards
Profile options	Hobbies, likes, live categories, support categories
Levels	Thresholds for user levels 0–20 and host levels 0–10
Payment credentials	**The bank account and Binance UID every seller pays into. Changing these takes effect immediately.**

> On that last page: the form **deliberately starts empty and shows what it actually found**, because the live IFSC, account number, holder name and Binance UID were previously written into the source file as defaults, shipping the real account in every bundle.

28.10 Department ownership — and where it is unclear

Evidenced ownership is in §4.3. **Eight places where ownership is genuinely undecided:**

1. **KYC and partner onboarding are split between App and Operations with no stated owner.** Both can navigate to the queue, but **only Ops holds the approval permission** — so App opens it and sees no Approve button. Agency registration is App-only; seller registration is App **and** Ops. Registering and approving sit in different departments, inconsistently by partner type.
2. **A partner's record belongs to App and Ops; its money belongs to Finance; both can open the same screens.** The pages admit Finance, the navigation shows it to neither. Which department is responsible for a partner record is decided nowhere.
3. **`chance` is owned by nobody.** Five of six screens are navigable by **all thirteen staff roles**; proposals are any team admin; activation is the owner; freezing is Ops; the money is Finance.
4. **The economy is read by Finance and Ops, configured by Finance, and the permission is held by Tech.** Tech holds a permission for a screen it cannot open.

5. **Broadcasts.** The navigation and the page say Marketing; the dashboard offers it to App and Ops and links them to a wall; the email composer's real authority is graded **by audience inside the API** across six admin roles plus the owner.
6. **The gift catalogue has no department.** It sits in the Economy group, is editable by three roles, is **readable by anyone who types the URL**, and its sibling editor lives under **Tech** on a different screen. Two departments edit one catalogue from two places.
7. **Team management has two screens that do the same job** — the application says so itself: *"This screen and the other now do the same job"*, adding that one should go and that is not a decision the page can make.
8. **App Configuration versus Tech.** All seven pages are owner-only, but the permission is held by three other roles. Business purpose of that permission unclear.

28.11 Known navigation and page disagreements

The navigation deliberately narrows five links below what the page itself allows, each with the reason cited inline — these are intentional. The live disagreements are:

- The dashboard links App and Ops staff to **Broadcasts**, which refuses them.
- The agency and seller pages admit Finance; the navigation does not. Finance reaches them by URL only.
- The moderation page admits an App member; the navigation does not.
- The tickets page admits a Tech member; the navigation excludes them because the Firestore read rule does not cover them — **and the dashboard quick action still offers it to them.**
- **The gift catalogue has no read gate at all.** Only an edit gate exists.

29. Cloud Functions and Scheduled Jobs

29.1 The 21 Cloud Functions

Function	Trigger	Business purpose	Money
`onRechargeApproved`	Recharge request updated	Seller coins down, buyer diamonds up	**Yes**
`settleGift`	Callable	The whole gifting transaction	**Yes**
`startCall` / `acceptCall` / `declineCall` / `tickCall` / `endCall`	Callable	The call lifecycle and its billing	**Yes**
`convertBeans`	Callable	Beans to diamonds, drawing down unreleased weeks	**Yes**
`purchaseBagItem` / `equipBagItem`	Callable	Cosmetics, charged in diamonds	**Yes**
`claimDailyReward`	Callable	The streak reward	**Yes**
17 party callables	Callable	Rooms, seats, invites, moderation	Gifts only
`generateAgoraToken`	Callable	The media token; **decides the identity and the publisher slot**	No
`activateHost` / `submitHostCapture` / `reviewHostVerification`	Callable	Host activation and verification	No
`requestAccountDeletion` / `recoverAccount` / `purgeMyAccountNow`	Callable	Account end-of-life	No
`syncPublicProfile` / `syncLivePresence`	User and room writes	The public projections the feed reads	No
`syncPublicSeller`	Seller write	The verified-reseller directory	No
`onAgencyApproved` / `onAgencyRegistered` / `onSellerRegistered` / `onSellerApproved`	Partner writes	Partner emails	No
`onAgencyResendEmail` / `onSellerResendEmail`	A field set by the CRM	**The manual retry for the four above**	No
`onApprovalApproved`	Approval request updated	Applies an approved request — **refuses money and host decisions**	No
`onPayoutApproved`	Weekly report approved	**Approval markers only**; no balance is written	No
`registerAgencyPartner` / `registerSellerPartner`	Partner created	Enqueues the partner into MongoDB	No
`toggleLike` / `syncMyRoles`	Callable	Social counters, role flags	No

29.2 Fourteen scheduled jobs

Seven appear on the jobs board. Seven appear on no monitoring board at all.

Registered, and visible on the jobs board

Job	Cadence (IST)	What it does
Transactional sweep	**Every 15 min**	Sends every transactional email a record says is owed and that the delivery log says has not gone
Alert scan	**Every 30 min**	Runs the sixteen detectors

Job	Cadence (IST)	What it does
Reconciliation	03:00 daily	Compares Firestore against the ledger. **Corrects nothing**
Daily reports	07:00 daily	Business summary, Finance worklist, exception report
Weekly business report	Monday 07:30	The owner’s profit and loss
Partner statements	Monday 09:00	A statement to every approved partner — **90 minutes after the owners see the week**
Monthly reports	1st, 08:00	Profit and loss, accounting pack, audit summary, and every partner’s monthly statement

> The reconciliation entry carries its own warning: it **was deployed and running nightly while absent from this table**, so it appeared on no jobs board and in no “job has never run” detection, and *“could have failed every night for a month in silence.”* Two canary tests caught it.

In no registry

Job	Cadence	What it does	Money
Weekly payout report generator	**Sunday 18:29 UTC**	Writes the weekly report, then **accrues commission for every approved agency**	**Yes**
Release beans for week	Monday 00:30 IST	Moves a closed week’s beans into withdrawable balance	**Yes**
Daily platform stats	00:00 UTC	Aggregates a day of gifts into a revenue document	No
Sweep stale calls	**Every minute**	Settles silent calls; marks unanswered ones missed	**Yes**
Sweep host levels	02:00 daily	Recomputes every host’s level	No
Purge deleted accounts	03:30 daily	Hard-purges past the grace period, **max 50 per run**	No
Sweep quick-login tokens	04:30 daily	Deletes expired login secrets	No
Expire cosmetics	**Hourly**	Expires rented items and clears the equipped pointers	No
Sweep empty party rooms	**Every minute**	Closes abandoned rooms	No

> The cosmetic-expiry job carries a instructive note: its original query form **matched nothing, forever, while appearing to run fine every hour.**

30. Notifications

30.1 Push — four messages, all data-only

Every push is a **data message**, never a notification message. The reason is recorded: a notification message is rendered by the operating system with no app code involved, so there would be no full-screen intent, no ringtone, and no suppression when the person is already in that chat.

Push	Trigger	Recipient	Notes
Incoming call	A call document created while ringing	The receiver	High priority, **30-second time to live**, direct-boot capable
Call status	A call leaves ringing	The receiver	Exists to **take the ringing notification down**
New message	A chat message created	The other party	**Suppressed if the recipient has blocked the sender.** 24-hour expiry
Live started	A room created and active	**The host's followers**	1-hour expiry

Tokens are one document per device. A legacy single-token field is read **only when the subcollection is empty**, specifically so the same token is not sent to twice.

There is no retry on any push. A dead token is deleted — that one device only, and the legacy field only if it names that exact token. Any other error is logged and the message is gone.

> **A documented cap.** The live-started fan-out is **hard-capped at 2,000 followers**, because beyond that the single invocation would read one token subcollection per follower inside one timeout, and a retry would repeat all of it. *"Until then the 2001st follower is not notified, which is a missed notification rather than a timed-out function."*

30.2 Email — two separate systems

A. The backend system

Every send writes a delivery row to MongoDB **before** the provider is called; if the row cannot be written, **nothing is sent**, because *"An email nobody can prove was sent is not a control."*

Recipients resolve **two ways and no third**: internal staff by role from MongoDB, and partner-facing from the partner's own record. There is deliberately **no hardcoded fallback** — an owner-bound report with no owner resolves to *nobody* and says so.

Fourteen delivery outcomes are tracked, including four that record a deliberate decision **not** to send, which is kept distinct from a failure. A provider's rejection keeps the provider's own words. Three outcomes are terminal, so a later "delivered" never overwrites a worse earlier fact.

Transactional email	Trigger
Staff invitation / access added	Inviting a staff member
Observer sign-in code	Observer login
Partner application received	**NO CALLER**
Partner approved	**NO CALLER**
Partner rejected	A 15-minute sweep. A variant records *"Rejection notice not sent — no reason on the record"*
Partner deactivated	A 15-minute sweep, printing commission still owed

Transactional email	Trigger
Payout approved / payout paid	A 15-minute sweep, with the amount and the UTR
Credit due soon / credit overdue	A 15-minute sweep; the overdue one carries **no dedupe key on purpose**, because repeat suppression handles cadence

> **Two fully-built emails with zero call sites.** Application-received and approved are implemented, templated and previewable, and nothing calls them. The emails a partner actually receives for those two events come from the **Cloud Functions**, which write no delivery record at all.

Eight scheduled reports go to owners, the observer and Finance on daily, weekly and monthly cadences, each with a dedupe key per recipient so *"one person's send failing or being withheld"* cannot block another's. Reports can be muted per address; **alerts cannot.**

> **There is no retry on backend email either, and the failure mode is worse than none.** The dedupe row is inserted **before** the provider call and is **not removed on failure**. On the next sweep the same key collides and the function returns early with *"Already sent for this event; nothing re-sent."* **A failed transactional email is permanently not sent.** Compensation is detection only: a critical alert to owners and the observer, and two screens that surface the rows. Nothing re-sends.

B. The Cloud Functions system

A **separate provider instance**, a hardcoded sender, inline HTML, and **no delivery record of any kind**. Nine emails: the weekly payout report to the superadmin, and registration and approval notices for agencies and sellers, plus four manual resends.

Failure handling differs per caller. If the key is unset it **logs and returns successfully**, so the caller cannot tell. On a provider error it rethrows. The resend functions **delete their trigger field before sending**, so a throw loses the request entirely.

> **The four resend functions ARE the retry mechanism** for the four automatic ones — a manual one, invoked by a staff member setting a field. There is no automatic retry, no delivery record, and no way to know a send failed other than reading function logs.

One piece of care worth naming: an approval email reads the identity providers on the account first, so it does not tell a Google-signup partner to use a password that does not exist, and falls back to a neutral sentence on any error because *"an Auth outage must not stop an approval email going out."*

30.3 In-app

Collection	State
The top-level notifications collection	**Four writers, no reader.** And the security rule gates reads on a different field name than any writer sets — so even if a reader were added, **every document would be denied to its own recipient.** The unread flag is written by four paths and flipped by none
The per-user notifications subcollection	One writer, **no reader**, and the code says so: *"It is written because removing it would lose the record, not because anything sees it."* Field names differ from the collection above again — three shapes for one concept
Broadcasts	**Works.** Written by the CRM, read by the app's Official tab
Announcements	**NOT FOUND** — declared in the rules, no reader, no writer
Admin alerts	One writer on a recharge failure, **no rule and therefore no reader**

> The broadcast story is worth recording. The CRM wrote one field name; the app has only ever read another — so **every broadcast ever sent rendered as an empty card**: an official chip, a timestamp, two blank lines. The **writer** was fixed rather than the reader, because a writer change ships with a web deploy while a reader change waits on a release *"that the people already holding the old build will never get."* Both sides now accept either name.

30.4 SMS and WhatsApp

NOT FOUND. No SMS provider of any kind; the phone sign-in code is delivered by the identity provider's own channel, not by ZIPPER. No WhatsApp integration. One comment notes that referral codes are "*printed, forwarded on WhatsApp and typed*" by humans — a remark about how codes travel, not an integration.

31. Audit

IMPLEMENTED, and append-only by construction: *“Nothing in this module updates or deletes. There is deliberately no such function — not even for owners.”*

31.1 What a record carries

Timestamp; the actor’s uid, name, role, team and a system flag; the action; the subject’s type and id; a before and an after; **a mandatory reason**; an optional link to the money card; and free metadata.

There is no request id, no IP address, no user agent and no trace id. The actor block is the only provenance.

System-actor discipline: a machine actor must have a name and a reserved uid prefix, or the write is **refused** rather than producing an entry that reads as a person. The prefix is reserved in both directions.

Transactional atomicity is opt-in. The session of the transaction performing the change may be passed, and the code is explicit: *“Without it the entry is not atomic with the change, which defeats the point — so callers that mutate money must always pass one.”* A caller that forgets gets a non-atomic entry silently.

Read scoping: owners and the observer see everything; a team admin sees their team; everyone else sees themselves.

31.2 The vocabulary

Sixty-two action names in the central registry, grouped as money, cards, partners, users, devices and moderation, partner operations, configuration, and access.

Thirty-one more exist only in local enumerations and never reach the central registry: nine partner operations, two appeals, twelve `chance`, eleven player-protection, three live-intervention and one broadcast action.

> **Five names are declared centrally and written from a duplicate local enumeration that happens to agree by value.** The same hazard was documented as having been removed elsewhere — *“the log was written from two enumerations that happened to agree, and the next person to add a value to one of them would have found out they disagreed by reading the audit log”* — and is still present here.

The CRM’s filter chips cover six groups and 38 action strings. **Roughly 55 names have no filter**, including every `chance` and protection action, every appeal, suspension and resumption, all four host application and verification actions, device bans, report actions, write-offs, commission accrual, and expenses. For a chance payout in particular — a credit of value to a player — **there is no way to ask “every chance payout this week.”**

31.3 What is not audited

Action	Audit
Configuration changes	**None.** Two actions are declared and **written by nothing**; the configuration that matters is edited by direct browser writes (§27, GAP-F007)
Withdrawal approve / reject / mark-paid	**None**, and `processedBy` is the literal string `'Admin'`
Salary approve / pay / reject / distribute	**None.** No actor, no reason
Coin purchase rejection	None — though it does capture an actor uid, which is better than the withdrawal path
Recharge approval	**None**, and real value moves
Device bans from the CRM	The backend path audits with before, after and a mandatory reason. **A parallel browser implementation writes the ban document directly with no audit, and unbanning deletes the document**, destroying the history rather than recording it
CRM role and partner-login approval	A browser write with no entry — and therefore outside the staff-access-changed alert
Everything on the Firestore side of the economy	Gifts, calls, conversions and rewards write Firestore subcollections that carry **no actor role, no reason, no before and after, and no card link**, and are not in the audit log

> **One logging bug.** Starting a review is recorded under the **same action name** as completing a verification. Separation of duties depends on knowing who verified a card; the audit log filtered to that action returns both “started looking” and “checked and passed”, with only the free-text reason to tell them apart.

32. Security

32.1 What the security rules actually govern

The Admin SDK bypasses them entirely. Every write by a Cloud Function or the backend is unconstrained. The rules therefore govern **the Android app and the CRM browser only** — which is what a rule is worth, and why closing a rule on a server-written collection costs the server nothing.

32.2 The layered model

Layer	Enforces
Firebase Auth	Who you are
`firestore.rules`	What a *browser or phone* may read and write
Backend route dependencies	What an API caller may invoke
Backend service predicates	The business rule, independently of the route
Card separation of duties	That no single person completes a money movement

Two places deliberately have **two guards where the narrow one wins**: freezing a `chance` player, where an Ops member fails the route and a Finance admin fails the service; and the chance money screens, gated narrower in the CRM than in the API *“because business-wide money is not Marketing’s.”*

32.3 Identity and session

Control	Behaviour
One live mobile session per account	A new device displaces the old one, **naming the device that took it**
Quick-login	A device secret in a platform keystore that survives clear-data and uninstall. A **button, never automatic**
“Sign out and forget this device”	Revokes the secret server-side so nobody can tap straight back in
Device bans	Keyed by device id, because that is the path the app reads. **Barring a handset locks out every account on it**
Guest accounts	Bound to one device secret. A second installation that cannot prove ownership is refused
Staff deactivation	Three stores updated, with dedicated audit actions for partial failure
Partner session revocation	A rejected or suspended partner’s live session is revoked
Screenshots	Blocked in release builds

32.4 Sensitive data handling

Data	Treatment
Host verification photographs	**Biometric data under India’s DPDP Act.** Signed URLs fetched only when a reviewer opens a case, held in component state, dropped on close, **never logged**. Readable by the host and reviewing staff — **not the agency**, because an agency that can read the verdict can work out how to get past it
Seller identity documents	On the private seller record only. The public directory is a **seven-field projection** with an explicit never-publish list

Data	Treatment
The company bank account	Owner-only to write. Previously **hardcoded in the source file**, shipping the real account in every bundle
Staff pay	Stripped from the staff list for every row that is not the caller's own
Gift history	Room-scoped. Previously readable by any signed-in account, which allowed reconstructing *"who had gifted whom, across the entire platform"*

32.5 Where security rules have not been compiled

> **GAP-S001 — The Firestore rules have never been compiled or emulator-tested in this environment.** No Firebase CLI, no rules-testing library, and the package registries are unreachable. The rules are asserted against by text-based tests, which are **not** equivalent to running them. Nine rounds of rule changes have never passed through a compiler. A staging project would compile them.

33. Configuration

33.1 Where configuration lives

Document	Holds	Who may write
`appConfig/economy`	**The commission slab tables**, the bean rate, the diamond price, gift rates, **the call split matrix**, the user call rate, minimum payout	Tech and the owner
`appConfig/paymentCredentials`	**The company bank account and Binance UID**	**The owner alone**
`appConfig/control`	Maintenance mode and message, force-update, minimum version, **ten feature flags**	Tech proposes; the owner saves directly
`appConfig/androidUpdate`	The self-hosted update channel	A publish script
`appConfig/dailyRewards`	The seven-day streak schedule	Owner
`appConfig/currencySettings`, `profileOptions`	Currency display, profile vocabulary	Owner
`levelConfig/host/levels`, `levelConfig/user/levels`	The two ladders	Owner
`contentConfig/*`	Level and profile-card content	Tech
`gifts`, `bagItems`, `rechargePackages`, `diamondPacks`, `coinPackages`	Catalogues	Tech, App, Ops / owner
`policies`	Privacy policy, terms, FAQ, per locale	App admin and owner, **with a separate Publish step**

33.2 How a change takes effect

Immediately, and without versioning, with three exceptions.

Mechanism	Where
Immediate, unversioned, unaudited	Everything in the table above
Propose-then-approve	App Control for Tech; the economy configuration for non-owners; the “changed details” partner queue
Immutable versioned, card-signed	`chance` **configurations only.** A change is a new version, raised as a card and signed by an owner
Draft then publish	Policies

> **Historical transactions do not use historical configuration values.** The exception is the commission statement, which reads **ledger entries** rather than re-deriving from today’s rates — *“that is what makes a March statement still say what it said in March.”* Everything else, including a gift’s receiver percentage and a call’s split, is read live at the moment of the transaction and is not recorded as a rate alongside the movement except where the transaction row happens to carry it.

33.3 The rate fallbacks

Rate	Fallback	Published meaning
Bean to diamond	**20 beans = 1 diamond**	*“Deliberate friction: without it, gift → convert → re-gift is a closed loop that mints diamonds nobody paid for”*
Bean to rupee	₹0.01 (one paisa)	1 lakh beans = ₹1,000

Rate	Fallback	Published meaning
Diamond pack retail	₹2,390 per lakh	Wholesale above retail is **clamped to retail**
User call rate	1,600 diamonds/min	Used only when neither party is a host
Host base call rate	375 beans/min	What an **unverified** host earns
Weekly minimum payout	₹2,000	
App-side withdrawal minimum	100,000 beans = ₹1,000	

34. Reporting

IMPLEMENTED, and unusually careful about what it does not know.

34.1 The conventions that run through every report

These are the most transferable design decisions in the system.

- **An unreadable figure is `None`, never zero.** A failed section says so in its own words, with every money column empty and a reason — *“That row shows unknown, not zero.”*
- **An empty result is “no data for this period”, never ₹0.**
- **Provisional is said above the number**, not in a footnote.
- **Medians, not averages**, for throughput. An uncompleted stage is a dash.
- **The weekly business report is written once and never updated.** Re-reading a week later gives the same figures, *“which is what makes it a record rather than a recomputation.”*
- **One agency’s failure produces one row that says so**, not a failed report.
- A host below an eligibility line **always gets the figure and the exact shortfall.**

34.2 The reports

Report	Cadence	Audience
Daily business summary	07:00	Owners, observer
Daily finance worklist	Daily	Finance admin
Daily exceptions	Daily	Owners
Weekly business report	Monday 07:30	Owners. Hosts, agency commission, recharges, seller credit, gross gaming revenue, a game margin model, a cannibalisation guard, leaderboards
Agency week	Weekly	**Eighteen columns**: billing, band, rate, commission, who it is paid via, open payment legs, and *why*
Host week	Weekly	Beans earned and cashed out, gifts, call minutes, eligibility **with the reason**
Partner statements	Monday 09:00, and monthly	Each approved partner, to their own address. Opening and closing balance, earned, paid, outstanding
Monthly profit and loss, accounting pack, audit summary	1st, 08:00	Owners, Finance, observer
Leaderboards	Weekly	Recharges, gifts received, lucky-gift spend. **No prizes**

34.3 What a partner sees of their own commission

The derivation screen is a notable piece of work: **nothing is recomputed in the browser.** Every sentence and every bound is written by the engine, and the screen lays them out — because *“a browser that does its own slab arithmetic can disagree with what is actually paid, and the CRM has already been down that road once.”*

It shows: a refusal state that prints the server’s own sentence and distinguishes it from a zero; the period and its half-open bounds; a provisional banner; **Step 1** — what the hosts cashed out, the rate applied,

and beans earned but not yet billing; **Step 2** — the floor, with the exact shortfall and how many more beans would reach band 1, or the headroom above it; **Step 3** — the band, the rate, the commission, and three verbatim server strings: the explanation, the rule and the rounding.

And then **the whole slab table, not just the matched band** — *“no private rates, and an agency should be able to see what the next band pays”* — with the matched row tagged “yours”, the next tagged “next”, and a footer naming exactly how much more billing the next band needs.

Finally, which hosts produced it, with a caption stating that the shares *“add up exactly — the leftover paise go to the largest remainders, so nothing is lost in the split.”*

35. Integrations and External Actors

35.1 Third-party services

Integration	Purpose
Agora RTC	All voice and video: calls, live, party rooms. Tokens minted server-side with a one-hour privilege
SendGrid	Every backend email, plus an inbound webhook for bounces, drops and spam reports
Firebase Cloud Messaging	Push. All four messages data-only
Firebase Storage	Verification photographs, application builds
Firebase Crashlytics and Analytics	Telemetry
Google ML Kit face detection	Liveness checks and the live-stream presence gate
TensorFlow Lite + Firebase ML	An on-device model downloaded at runtime
Google Play Services Block Store	Keeps the quick-login secret where it survives clear-data, uninstall and device transfer
Play Services Location	Reverse geocoding at profile setup
GPUImage, CameraX, Media3, Lottie	Filters, capture, playback, gift animations
Credential Manager + Google ID	Google sign-in
Apache Commons Compress	Exactly one class, for binary update patching

Payout rails are manual, not integrated. The payment mode is free text; UTR, transaction reference and proof image are captured by hand.

35.2 Machine actors

Three service credentials, each deliberately narrow, presented as a header, each resolving to a reserved system identity:

Credential	Can do
Weekly accrual	Create agency commission liability — **two routes only**
Reporting	Send reports. *"the worst a leaked reporting key can do is send a report"*
Partner registration	Enqueue a new partner

Three automated actors with no credential: player protection, the suspension sweep, and a per-caller `chance` identity.

And **one external actor that can move money**: a third-party game provider settling a round (§18.6).

35.3 A whole distribution channel: self-hosted updates

DISCOVERED — and significant. The app side-loads its own updates outside the app store.

A publish script reads the version out of each build binary rather than taking a flag, uploads per-architecture builds to storage, and writes the update document. The app carries a **from-scratch binary**

patcher, so an incremental update ships single-digit megabytes instead of about ninety. The downloader deliberately does **not** use the platform download manager — which *“can be silently disabled by battery-saver apps and still returns an id”*, observed three times — verifies a hash against the published digest, and hands the file to the system installer.

> **The manifest’s own comment states the constraint:** *“GOOGLE PLAY FORBIDS AN APP UPDATING ITSELF OUTSIDE PLAY.”* The feature and its permission exist for hand-shared team builds, and the comment states that a store submission needs that manifest line removed as well as the build flag flipped. **Both release and debug build types currently enable it.**

36. Data Model

36.1 MongoDB — the business of record

Collection	What it represents in business terms
`staff`	Every internal person: role, department, employment details, pay, active flag
`partners`	Every agency, sub-agency and coin seller as a **business**: status, kind, parent, credit position, totals
`ledger`	**Every balance movement the business can account for.** Holder, currency, amount, action, balance before, balance after, reason, idempotency key, full metadata
`balances`	The current balance per holder and currency, kept in step with the ledger inside one transaction
`cards`	Every request for money to move, and where it is in the five stages
`audit`	The append-only record of who did what, when, to what, and why
`sessions`	Issued session tokens, revocable
`rates`	Versioned economy rates
`partner_sales`	Seller-to-buyer sales
`subagency_payouts`	The two-leg payouts, with each leg's own status
`observer_otp`	One-time codes for the observer
`email_deliveries`	Every email, with a unique dedupe key and fourteen possible outcomes
`alert_state`	Suppression records — the answer to *"I was not told"*
`chance_mechanics`, `chance_configs`, `chance_rounds`, `chance_exposure`, `chance_player_flags`	The games control plane
`weekly_business_reports`	Immutable weekly records, keyed by ISO week

36.2 Firestore — what the app lives in

97 collections. Grouped by what they are for.

Group	Collections
Identity	`users`, `publicProfiles`, `crmUsers`, `verifiedIdentities`, `identityResolutions`, `guestDeviceBindings`, `quickLoginTokens`, `fcmTokens`, `bannedDevices`
Social	`followers`, `following`, `liked`, `likedBy`, `blocked`, `conversations`, `messages`, `comments`, `reports`, `warnings`, `appeals`
Live and calls	`liveRooms`, `viewers`, `listeners`, `calls`, `callHistory`, `pkInvites`, `partyRooms`, `seats`, `seatRequests`, `seatInvites`
Gifts	`gifts`, `giftTransactions`, `giftEvents`, `giftShowcase`, `giftPairs`, `giftsSent`, `giftsReceived`, `giftLottieData`, `topFans`, `giftIdempotency`
Economy	`diamondTransactions`, `beanTransactions`, `weeklyEarnings`, `weeklyStatements`, `dailyStats`, `ownedItems`, `bagItems`, `levels`, `levelConfig`, `platformRevenue`, `platformStats`
Partners	`agencies`, `agencyHosts`, `commissionLedger`, `sellers`, **`publicSellers`**, `rechargeRequests`, `rechargeHistory`, `coinPurchaseRequests`, `coinPurchases`, `diamondLending`
Hosts	`hostApplications`, `hostVerification`, `hostPayouts`, `withdrawalRequests`, `salaryRequests`, `weeklyPayoutReports`
Operations	`feedbackTickets`, `supportTickets`, `crmSupportTickets`, `techTickets`, `opsTickets`, `approvalRequests`, `kycSubmissions`, `seatRequests`
Configuration	`appConfig`, `contentConfig`, `paymentConfig`, `policies`, `rechargePackages`, `diamondPacks`, `coinPackages`
Comms	`broadcasts`, `announcements`, `notifications`

36.3 Key relationships

```
crmUsers  <-->  staff          one internal person, two records
crmUsers  <-->  partners       one partner login, one business record
                               (a dual-capability login holds a roles[] array)

agencies/{id}
   |  parentAgencyId  -->  agencies/{parent}     a sub-agency
   |  /agencyHosts/{hostUid}                     the roster, server-written
   v
users/{uid}  isHost: true, agencyId
   |  /weeklyEarnings/{weekKey}      the bean bucket, released a week later
   |  /diamondTransactions           what they spent
   |  /beanTransactions              what they earned
   |  /dailyStats/{date}             hosts only
   |  /warnings, /blocked, /topFans, /giftShowcase
   v
publicProfiles/{uid}       the projection the feed reads — never `users`

sellers/{id}  coinBalance
   v
publicSellers/{id}         seven fields only — the app's reseller directory
```

36.4 Collections declared and never used

Collection	State
`crmSupportTickets`	A full read/create/update/delete rule surface. **No reader, no writer, no type anywhere.**
`announcements`	Declared. No reader, no writer.
`kycSubmissions`	Declared, and **counted on a dashboard tile.** No writer exists, so **the tile always reads zero.**
`commissionLedger`	A payment history per agency. **No writer anywhere in four codebases.**
`rechargeHistory`	Coin-load records. **No writer.**
`coinPurchases`	A legacy alias. **No reader and no writer.**
`giftsSent` / `giftsReceived`	Carry rules, including a create rule open to any signed-in account. **Never written** — *“so both sections were empty on every profile.”*
`hostKyc`	Referenced in rules. **No such collection exists.**

37. State Machines — Consolidated

Only states that actually exist are listed.

Entity	States	Notes
User account	`active`, `suspended`, `banned`, `recovery_pending`, `permanently_deactivated`	Declared **twice** — five values in one language, three in the other. `isBanned` is the gate everything reads
Host application	`pending`, `approved`, `rejected`	**One writer of the decision**; two older writers were deliberately closed
Host application KYC	`submitted` (born), `approved`, `rejected`	`pending` is in the type and never carried
Host verification	`pending_capture`, `pending_manual`, `verified`, `rejected`	`pending_capture` is deliberately **not reviewable**. Replacing the agency photo resets it unless already verified
Agency (Mongo)	`pending`, `approved`, `rejected`, **`suspended`**	
Agency (Firestore)	`pending`, `approved`, `rejected`	**No `suspended`** — yet it is written there anyway
Agency KYC	`pending`, `submitted`, `approved`, `rejected`	Approved-without-documents is **left at pending on purpose**
Coin seller	Same four, and the status field is **optional**	Many records predate it and are decided by an approval flag
Seller credit standing	`none`, `current`, `due_soon`, `overdue`, `written_off`	**Derived, never stored.** `written_off` is **unreachable**
Staff	A single active flag	**No invited state**; born active. **No reactivation function**
Money card	`requested`, `under_review`, `awaiting_signoff`, `approved`, `executed`, `rejected`, `failed`	**`failed` is a dead end** — rejectable, never retryable
Approval request	`pending`, `approved`, `rejected`	**A technical failure and a human refusal are the same value**; a separate flag is the only discriminator, and it is not in the CRM filter
Withdrawal	`pending`, `approved`, `rejected`, `paid`	**Nothing creates one**
Salary request	`pending`, `approved`, `paid`, `distributed`, `rejected`	`rejected` was added because a refusal used to write `distributed` and was indistinguishable from a completed payment
Recharge request	`pending`, `approved`, `rejected`	**`approved` is not terminal** — a failure writes it back to pending so the retry can fire
Coin purchase request	`pending`, `approved`, `rejected`	**`approved` is declared and unreachable.** Approval moved to the card board; nothing writes it back. A seller whose purchase was approved sees `pending` forever
Sub-agency payout	`pending`, `parent_paid`, `closed`	Two legs, each with its own sub-status, written together
Call	`ringing`, `connected`, `ending`, `ended`, `missed`, `declined`	Plus a finer seven-value reason vocabulary
Live room	A single active flag	Plus a host-mute axis the app does not read
Party room	A single active flag	Seat state is **four independent fields, deliberately not collapsed**
Chance round	`committed`, `playing`, `recorded`, `settled`, `void`	*"A crash leaves it at `playing`"*
Chance config	`draft`, `live`, `superseded`, `withdrawn`	
Tickets	**Six collections, four vocabularies**	See §38
Appeal	`pending`, `upheld`, `overturned`	Four consequence types, each reversing differently
Email delivery	Fourteen outcomes	Including four kinds of deliberate non-send, kept distinct from failure

37.1 The ticket vocabularies

Collection	Values	Declared where
`supportTickets`	open, in_progress, replied, closed	The CRM library

Collection	Values	Declared where
`feedbackTickets` — both CRM screens	open, assigned, in_progress, resolved, closed	Both pages, identically
`feedbackTickets` — the shared type	open, in_review, resolved	**A third vocabulary agreeing with neither screen**
`feedbackTickets` — the app	open, in_progress, resolved rendered	**A fourth**
`techTickets`	open, in_progress, review, done	
`opsTickets`	open, resolved	The update function takes **any string**
`reports`	pending, reviewed, actioned, dismissed	`reviewed` **is written by nothing** — a permanently empty tab

The feedback collision was found and fixed with the scar tissue left visible: two screens read one collection and disagreed about what a status was, so a status filter ran against a vocabulary that did not contain the value. Both screens now carry an identical five-value list and an identical legacy-alias map, normalising on read and migrating on write. **The shared type was never updated**, and **the app is a fourth vocabulary** — a ticket moved to `assigned` has no colour in the app that filed it.

> **The deeper split was not reconciled.** A security rule asserts that two collections *"point at the same logical data; Android writes to one, the CRM reads from the other."* That is not what the code does. The app writes **and** reads one; a second app path reads the other; the CRM's ticket screen reads and updates the second; two other CRM screens read and update the first. **Nothing creates a document in the second collection at all** — so the reply, status, assignment and routing functions all operate on a collection with no producer, and **a ticket filed from the app can never receive a reply.**

38. Feature Status Classification

A. Implemented and active

Identity and onboarding · phone, Google, guest and quick-login sign-in · profile setup · discovery · profiles, likes, blocks, reports · chat · **calls** with tick billing · **live streaming** · **party rooms** with seventeen callables · **gifting** · cosmetics and daily rewards · host activation and verification · host levels · **the progressive commission engine** · weekly bean release · host withdrawals on the card path · the five-stage money board with twenty-two card types · lending · owner capital · expenses · **the public seller directory** · moderation, bans, warnings, appeals · the CRM's 78 routes · the kill switch · fourteen scheduled jobs · sixteen alert detectors · eight scheduled reports · partner statements · the append-only audit log · self-hosted application updates.

B. Implemented but disabled, or not reachable

Feature	Why
The `chance` subsystem	Complete server control plane. **No Android code path reaches it.** Status LIVE/NOT LIVE is unknown (§18.8)
Host muting mid-stream	Written and audited by the CRM. **The app reads no such field** — *"Until the app honours it, the host is still audible"*
The `events` feature flag	Default enabled, **gates nothing that exists**
Two partner emails	Fully built, templated, previewable, **zero call sites**

C. Partially implemented

Feature	What is missing
PK battles	Full interface, queue, timer and score model. **The security rules forbid every cross-room write it depends on, so PK never starts**
The salary flow	Firestore-only. No ledger movement, no audit, no actor, no backend function, and the card it is counted as is never raised
The withdrawal queue	A reader and three writers, and **no producer**
Reconciliation	Detects divergence nightly, **corrects nothing**
Sub-agency as an entity	Behaviourally separate, structurally a field. Depth unguarded
Events	Three unconnected fragments
The in-app notification collections	Four writers, **no readers**, and a rule reading a field no writer sets

D. Discovered — business meaning unclear

Collected in §39.

E. Business requirement, implementation not found

Requirement	State
Owner deactivation or deletion by the Observer	**Not found**
The department-admin vacancy rule	**Not found**
Host transfer between agencies	**Not found**
The agency-suspension cascade and its notifications	**Not found**
Payment gateway, callbacks, webhooks	**Not found**
Refunds, reversals, chargebacks	**Not found**
Commission adjustment or reversal	**Not found**
A second bean-count payout condition	Supported; **not configured**
Payout hold	Read in four places; **written by nothing**

F. Legacy, dead or unused

The Games tab placeholder · video cards, minted and unspendable · the user XP ladder, with **nothing writing its input** · `totalSubAgencies`, always zero · six declared-and-unused collections (§36.4) · two dead gift subcollections · a hardcoded privacy policy reachable from nowhere · a hardcoded terms document reachable only from the login screen · the deleted raffle-entry design · a dead security clause comparing two different identifier spaces · a documented sub-agency payment subcollection with no writer · the `payout.execute` ledger action, which no service writes.

G. Unknown

Whether `chance` is live · whether a salary request ever becomes an executed card · whether the `payout.execute` action is ever written in production · the live configuration values at the time of reading · whether the withdrawal queue is pre-wiring or an abandoned path · what the three CRM superadmin security questions are for · whether the gender-based call split is current intended policy.

39. Discovered — Business Meaning Required

Each item is something the system does that no business description provided covers. The system's behaviour is stated; the purpose is not inferred.

39.1 The `chance` subsystem

What was found. ~6,900 lines implementing a regulated games control plane: commit-reveal provably-fair draws, immutable owner-signed configuration versions, RTP bands, three daily payout ceilings plus a reserve enforced by compare-and-swap, a third-party settlement API, and a full responsible-gambling system with self-exclusion, cooling-off, loss limits, delayed limit increases, session reminders and three auto-detected harm patterns. Stakes and payouts are **real ledger movements**.

What we know. It works. Four alert detectors watch it. Six CRM screens operate it. The money is wired to the real ledger.

What we do not know. Whether it is live. Whether the lucky gift in the catalogue is meant to be settled by it. Whether the platform daily payout ceiling and both default loss limits are intended to remain at zero.

Why it matters. It is the largest and most carefully built subsystem in the product, it moves money, and the business has not described it.

39.2 The gender-based call split

What was found. A 3×3 table indexing the earner's percentage by the payer's gender and the earner's gender. Male pays female gives the earner 50%; female pays male gives 40%; everything involving the third option is a uniform 40%. **Anything unrecognised — blank, missing, a guest who never completed setup — resolves to the third row and column.** Gender is set once at profile setup and locked.

What we do not know. Whether this is current intended policy.

Why it matters. It is a pricing rule based on a protected characteristic, locked at signup, with legal and product implications in several jurisdictions.

39.3 Video cards

What was found. A consumable counter, minted only by the daily reward, with a My Bag tab describing it: *"Each card lets you start a free random call."*

What we know. It is created with no transaction row of its own.

What we do not know. Nothing in the product spends them, and there is no random-call feature anywhere in the source.

39.4 The user XP ladder

What was found. A twenty-rung user level ladder with thresholds, perks and a rules card, all driven by an experience-point field.

What we know. The CRM states it flatly: **“NOTHING ON THE PLATFORM WRITES IT.”**

What we do not know. Whether the ladder is intended to be driven by lifetime diamond spend, which *is* written, or whether the input is simply missing.

39.5 The three superadmin security questions

What was found. Every superadmin login requires answering three personal questions — full name, phone, nickname — set on first login, three attempts, then a forced sign-out.

What we do not know. What threat this addresses, and whether the answers are stored in a form that makes it meaningful.

39.6 Self-hosted application updates

What was found. A complete out-of-store distribution channel with a from-scratch binary patcher. **The manifest’s own comment states that the app store forbids it.** Both build types enable it.

What we do not know. Whether distribution is intended to remain outside the store, or whether this is a bridge to a store release.

39.7 Two profit formulas

What was found. A margin report and a profit statement, computing different things with different conventions.

What we do not know. Which is the figure the business considers its profit.

39.8 `platformCut` mixes two units

What was found. Both the call and the gift settlement write a platform-revenue row whose cut is **gross diamonds minus payout beans**. With 20 beans to a diamond, those are not the same unit.

What we do not know. Whether the figure is meant as a diamond amount or a notional ratio. Nothing states it either way.

39.9 Other discovered items

Item	What we do not know
PK scoring by gift **quantity**, not value	Whether one 5,000-diamond gift should score the same as one rose
Party seat gifting earns beans for a non-host with **no level progression**	Whether that is intended
Rose ×1 pays the host nothing, because the floor is per transaction	Whether the batching effect on cheap gifts is intended
The lucky fallback is **50% in code, 5% in the CRM**	Which is correct
Agency and seller records approved **without documents are left at KYC `pending`**	Confirmed as intended? The mechanism is deliberate
`commissionConfig` holds a Binance fee and a government tax percentage that are **never deducted anywhere**	What they are quoted for
The eligibility window is a **rolling 7 days**; the commission window is the **ISO week**	Confirmed as intended? The code says *“The two do not overlap and are not meant to”*

Item	What we do not know
Commission is computed in **UTC** while the business runs on IST	Intended?

40. Gap Register

A gap is **not automatically a bug.** Some of these may be intentional. None is resolved here.

ID	Area	Current behaviour	Expected behaviour	Severity	Needs business confirmation
GAP-F001	Finance	Two financial systems; app-side movements never reach the ledger; nightly detection, no correction	One auditable source of truth	**Critical**	No — a design decision
GAP-F002	Finance	Recharge approval has **no idempotency key** and `approved` is deliberately re-firable	A double approval must not transfer twice	**Critical**	No
GAP-F003	Finance	A seller's coin balance is direct-set with **no per-movement record in either store**	Every coin movement traceable	**Critical**	No
GAP-F004	Finance	Withdrawal approve/reject/pay are browser writes recording `processedBy: 'Admin'`, with no audit, repeatable, unlinked to the card	An actor, an audit entry, and a link to the card	**High**	No
GAP-F005	Finance	The withdrawal queue has **no producer**	Either a producer or removal	Medium	**Yes**
GAP-F006	Finance	**No payment gateway, callbacks, webhooks, refunds or chargebacks**	A gateway, if money is to enter directly	**High**	**Yes**
GAP-F007	Security	The **critical** bank-details-changed alert reads actions nothing writes; configuration is edited by unaudited browser writes	Changing the company's receiving account must audit and alert	**Critical**	No
GAP-002	Governance	No path to deactivate or delete an owner	Only the Observer may	**High**	No
GAP-003	Governance	No department-admin vacancy state	Members report to an owner until a new admin is appointed, then revert	**High**	No
GAP-004	Partners	**No host transfer workflow**	App Admin initiates, Owner decides, history preserved both sides without double counting	**High**	No
GAP-005	Partners	**`payoutHold` is written by nothing**	A host can be put on payout hold	**High**	No

ID	Area	Current behaviour	Expected behaviour	Severity	Needs business confirmation
GAP-006	Partners	**No agency-suspension cascade or notification**; a suspended parent makes sub-agency commission permanently unpayable with no alert	Hosts and sub-agencies transferable, all parties notified	**Critical**	No
GAP-007	Finance	The salary request and the salary card are **unconnected**	One flow	**High**	**Yes**
GAP-008	Audit	**No actor and no reason on any salary transition**	Every money decision names a person and a reason	**High**	No
GAP-SA-001	Partners	Sub-agency **leg 2 moves no money**; the sub reads zero from leg 1	The sub's receipt is recorded	**Critical**	No
GAP-SA-002	Partners	**Sub-agency depth is unguarded**; UI-only prevention; two competing definitions of "is a sub-agency"	Exactly one level	**Critical**	No
GAP-SA-003	Partners	No parent cascade; an orphan sub is detected after the fact	Prevention, or a transfer path	**High**	No
GAP-SA-004	Security	A dead agency read clause comparing a document id to an auth uid; two rule comments promise clauses not in the body	Rules match their comments	Medium	No
GAP-SA-005	CRM	`totalSubAgencies` is always zero and is rendered as a count	A correct count, or removal	Medium	No
GAP-S001	Security	**The Firestore rules have never been compiled or emulator-tested**	Compiled and tested before production	**Critical**	No
GAP-T001	Operations	**Six ticket collections, four vocabularies; one has no producer; a ticket filed from the app can never be replied to**	One coherent support flow	**High**	**Yes**
GAP-N001	Notifications	**No retry on any push or email**, and a failed transactional email is **permanently** not sent because the dedupe row survives the failure	A retry, or an explicit decision not to have one	**High**	No
GAP-N002	Notifications	The in-app notification collections have **four writers and no readers**, and the rule reads a field no writer sets	Either read them or remove them	Medium	**Yes**
GAP-A001	Audit	~55 audit actions have **no CRM filter**, including every chance payout	Filterable	Medium	No
GAP-A002	Audit	Starting a review logs under the **same action** as completing a verification	Distinguishable	Medium	No
GAP-A003	Audit	Thirty-one actions live only in local enumerations; five collide with the central registry by value	One registry	Medium	No

ID	Area	Current behaviour	Expected behaviour	Severity	Needs business confirmation
GAP-C001	Configuration	Configuration is **immediate, unversioned and unaudited**, except `chance`	Versioned and audited, at least for money-bearing values	**High**	**Yes**
GAP-L001	Live	PK battles cannot start — **the rules forbid every write the feature needs**	Either server-side PK or removal	**High**	**Yes**
GAP-L002	Live	A **host disconnect does not stop billing**; the payer keeps paying with no server-side presence check	Billing stops when the earner leaves	**High**	No
GAP-L003	Live	Ending a stream leaves comments, gifts, viewers and both counters in place	Cleanup, as party rooms do	Low	No
GAP-L004	Live	The viewer counter is **writable by any signed-in user** and is not reconciled with presence documents	One trustworthy count	Medium	No
GAP-M001	Mobile	**No maintenance-mode gate in the app**, although the CRM has the switch	The switch takes effect	**High**	No
GAP-M002	Mobile	**Three competing legal documents**: a hardcoded terms reachable only from login, a hardcoded policy reachable from nowhere, and CRM-managed documents in Settings	One source	Medium	**Yes**
GAP-M003	Mobile	**Follow is live-only**; no profile button; unfollow never called; a push channel describes a feature with no entry point	A follow model	Medium	**Yes**
GAP-M004	Mobile	The Games tab is a placeholder while Terms, the Level rules and two wallet labels describe games as live	Consistency	Medium	**Yes**
GAP-M005	Mobile	“Go Live” is offered to everyone and refused to non-hosts; the agency dashboard says “Apply from your profile” where no apply flow exists	Consistency	Low	No
GAP-D001	Data	Six declared collections with no reader or writer; a dashboard tile that always reads zero	Removal, or implementation	Low	No
GAP-D002	Data	**Agency status vocabularies differ between stores**; `suspended` is written into a Firestore field whose type excludes it and no badge renders	One vocabulary	Medium	No
GAP-D003	Data	`written_off` credit standing is **unreachable**	Reachable, or removed	Low	No
GAP-D004	Data	A coin purchase approved on the card board leaves the Firestore request at `pending` **forever**, so the seller never sees approval	The seller sees the outcome	**High**	No

ID	Area	Current behaviour	Expected behaviour	Severity	Needs business confirmation
GAP-D005	Data	A technical approval failure and a human refusal share one status value	Distinguishable	Medium	No
GAP-X001	Finance	**No commission adjustment or reversal path exists**, and the idempotency key blocks re-accrual at a corrected figure	A correction path	**High**	**Yes**
GAP-X002	Finance	**No capital failure path** — settlement passes success unconditionally	A failed bank transfer can be recorded	Medium	No
GAP-X003	Finance	A **failed card is a permanent dead end** — rejectable, never retryable	Stated as intended; confirm	Low	**Yes**

41. Edge Cases — Documented Behaviour

Where the system has an answer, it is given. Where it does not, that is the finding.

41.1 People

Case	Behaviour
Agency suspended with active hosts	**Nothing happens to the hosts.** No cascade, no notification, no transfer (GAP-006)
Agency suspended with sub-agencies	The subs keep accruing commission that **can no longer be paid**, because a payout requires an approved parent. Detected in the weekly report after the fact
Parent agency deleted	The sub's parent reference dangles. The report classifies it: *"there is no main agency to route this commission through, so it cannot be paid"*
Department admin removed	**No vacancy state** (GAP-003)
Owner deactivated	**No path exists** (GAP-002)
Host transferred while earning	**No transfer workflow** (GAP-004)
User deleted holding a balance	Soft-deleted for 30 days, then purged. **Seven subcollections are retained**, including both transaction histories, because *"a payment question can still be answered later"*
Seller suspended holding coins	Withdrawn from the public directory. The inventory and credit position persist on the record
Duplicate role assignment	A second capability is a **new business**, not a promotion; it files an application a reviewer must approve

41.2 Money

Case	Behaviour
Duplicate recharge approval	**Transfers twice.** No idempotency key (GAP-F002)
Duplicate gift	Guarded by a claim document; the stored result is returned and nothing is charged. **The live path mints a fresh key per tap deliberately**, so two deliberate taps are two gifts
Gift with insufficient balance	Refused before anything moves, **quoting both numbers**
Failed withdrawal at step 2	The card is left **failed**, flagged as needing manual correction, with the Firestore debit already applied
Partial payment	No concept — payment is out of band
Refund	**No path exists**
Reversal of commission	**No path exists** (GAP-X001)
Negative balance	Refused by the ledger unless explicitly permitted; that permission is used **once**, for a host bean withdrawal
Concurrent transactions	Firestore transactions and Mongo sessions; the `chance` ceilings use **atomic compare-and-increment**
Duplicate webhook	**No webhook exists**
Settlement mismatch	A **critical** alert detector exists and reads a collection nothing writes
A user's cash payment to a seller	**Entirely outside the system.** The only record is a screenshot and the seller's approval

41.3 Calls

Case	Behaviour
Payer runs out mid-call	Affordable ticks charged, status moves to ending, the app closes the call
Host disconnects	**The payer keeps being charged.** No server-side presence check (GAP-L002)
Payer disconnects	Ticks stop; the watchdog settles two minutes later **including the silent gap**
Both disconnect	The same watchdog path, with no party check
Server dies after a charge	One transaction per tick: all or nothing. A lost response is covered by the tick counter
An unanswered call left ringing	Marked missed after 45 seconds, inside a transaction that re-checks the status so a simultaneous accept wins cleanly. **This exists to prevent a stale call re-ringing days later and billing a vanished caller**
Billing succeeds but the call record fails	Both are in the same transaction
A hang-up that fails	**Ticks deliberately keep running** until the server agrees, then the person is told they are still being charged

41.4 Live and party

Case	Behaviour
Host ends a live unexpectedly	The flag is written; a trigger updates the projection. **Comments, gifts, viewers and both counters remain**
Host loses network	**No server-side timeout on a live room.** The face-detection countdown runs on the host's own device and cannot fire if the device is gone
Viewer stays connected to a dead room	The media token function refuses a token for an inactive room
Gift during shutdown	The transaction either commits or not; the room mirror is skipped without complaint if the room is gone
Party room abandoned	**Two fuses**: 2 seconds for an emptied room, 10 minutes for one never occupied
Party room shrunk under an occupant	The seat is deleted and the count decremented
Wrong private password	The attempt counter increments **and the error still throws** — deliberately not a lockout
Re-join after the app was killed	No password re-prompt if the presence row survived

41.5 Chance

Case	Behaviour
Round interrupted	**Left at `playing`** — stated in the code
Duplicate request	Keyed per round for both the stake and the payout
Provider figure disagrees	**Recomputed on this side and the settlement refused**
A ceiling exceeded	The per-round ceiling caps silently; the daily budgets and the reserve **trim, withhold, and auto-suspend the mechanic**
Configuration changed during a round	A configuration version is **immutable**; a change is a new version
Protection unavailable	**Fails closed** — play is blocked

41.6 Data

Case	Behaviour
Orphaned sub-agency	Detected in the weekly report, named, and counted. Not prevented
Missing parent	The payout refuses; the commission accrues unpayably
Duplicate documents	A sparse unique index on the business id; a documented trap where a missing id makes a second record creatable
Inconsistent balances	Compared nightly, **not corrected**; the policy is that Mongo is right
Missing ledger entries	The normal case for everything the app does (§19)
A malformed slab table	**Raises** rather than falling back, with a distinct status code
A missing rate	Falls back to a documented value, marks the figure **provisional**, and **refuses an unattended accrual**

42. Open Questions

Only questions that **cannot** be answered from the code appear here. They are grouped and left for Phase 2.

Business

1. **Is `chance` live today?** If not, is it intended to launch, and is the lucky gift in the catalogue meant to be settled by it?
2. **Is the gender-based call split current intended policy?**
3. **Video cards** are minted and unspendable. An unbuilt feature, or dead?
4. **The user XP ladder has no input.** Should it be driven by lifetime diamond spend, which is written?
5. **Is distribution intended to remain outside the app store?**
6. **The Games tab is a placeholder** while Terms, the Level rules and two wallet labels describe games as live. Which is the intent?

Finance

7. **Which profit formula is *the* profit?** The margin report and the profit statement compute different things.
8. **What are the Binance fee and government tax percentages quoted for**, given that neither is deducted anywhere?
9. **Should a commission correction be possible?** No adjustment or reversal path exists, and the idempotency key blocks re-accrual at a corrected figure.
10. **Is `platformCut` meant to be a diamond amount?** It currently subtracts beans from diamonds.
11. **Should the second payout condition — "one lakh beans in a week" — be configured?** The mechanism exists and is unset.
12. **Is the salary flow meant to produce a card?** The reports count executed salary cards and nothing raises one.
13. **Should the commission window move to IST**, matching the rest of the business calendar?

Operations

14. **Which ticket collection is the support flow?** Six collections, four vocabularies, one with no producer, and a ticket filed from the app cannot be replied to.
15. **Which department owns KYC and partner onboarding?** Both App and Ops can navigate; only Ops can approve.
16. **Which department owns the gift catalogue?** Two departments edit it from two screens, and it has no read gate.
17. **Which of the two team-management screens should remain?** The application says one should go.
18. **Is the withdrawal queue pre-wiring, or abandoned?**

Technical

19. **Should configuration changes be versioned and audited?** Only `chance` is today.
20. **Should push and email have a retry?** Neither does, and a failed transactional email is permanently not sent.
21. **Should PK battles be moved server-side, or removed?** The rules forbid every write the feature needs.
22. **Should the in-app notification collections be read, or removed?** Four writers, no readers.

Compliance

23. **Should the `chance` default loss limits and platform payout ceiling remain at zero?**
24. **Which legal document is authoritative** — the one a person agrees to at signup, or the one Settings shows them afterwards?
25. **Is the 30-day deletion grace and the seven-subcollection retention the intended data-retention position?**

Product

26. **How should a host move between agencies**, given that historical earnings must follow the host while the old agency keeps its own record without double counting?
27. **What should happen to hosts and sub-agencies when an agency is suspended?**
28. **Should a parent agency earn from its sub-agencies' production?** Today it earns nothing. This is the question that determines whether the commission engine needs a roll-up it does not have.

43. Phase 1 Investigation Plan

Phase 0 is description. Phase 1 is validation. This is what Phase 1 should establish, in an order that respects dependency.

Step	What to establish	Why it is first
1	**Read the live configuration document.** Every slab, rate and percentage in this document is the documented fallback. The live values were not read	Every financial figure in Phase 1 depends on it
2	**Determine whether `chance` is live**, and whether any round has ever been staked	It moves money and is the largest unknown
3	**Survey the two financial systems against each other** on real data — how far apart are they, and in which direction	Sizes GAP-F001
4	**Count the exposure on GAP-F002 and GAP-F003** — how many recharge approvals have re-fired, and whether any seller inventory is unexplainable	Establishes whether a critical gap has already cost money
5	**Survey sub-agency depth and orphans** on real data — does a chain deeper than one level exist today, and are there subs with no parent	Sizes GAP-SA-002 and GAP-SA-003
6	**Compile a real permission matrix from production role records**, not from the table	The table and the screens disagree in eight places
7	**Compile and emulator-test the security rules** in a staging project	GAP-S001; blocks any further rule change
8	**Reconcile the four ticket vocabularies** against real documents	Determines whether support has ever worked end to end
9	**Confirm or correct each Gap Register entry's severity** with the business	Severity is a business judgement, deliberately not made here
10	**Answer §42**	Phase 2 cannot begin without it

Nothing in Phase 1 should modify production data. Steps 1, 3, 4 and 5 are read-only surveys, and a dry-run survey script already exists for the phone-identity equivalent, which can serve as the pattern.

44. Completeness Check

What this document covers, and what it does not.

Area	Covered	Confidence
Every codebase inspected	Yes	All four
Android flow, screen by screen	Yes	49 screens
CRM flow, route by route	Yes	78 routes
Backend services and routers	Yes	15 routers, 23 services
Cloud Functions	Yes	21
Scheduled jobs	Yes	14, including 7 in no registry
Database, both stores	Yes	97 Firestore collections, 16 Mongo
Every role documented	Yes	16, plus 3 things that are not roles
Every role lifecycle	Yes	User, host, agency, sub-agency, seller, staff, owner, observer
Major entity lifecycles	Yes	27 state machines
Live, calls, party, gifting	Yes	With arithmetic and worked examples
Events	Yes	As three unconnected fragments
Chance and player protection	Yes	Status unknown, stated as such
Wallets and ledgers	Yes	Including the two-system finding
Recharge and payments	Yes	Including the absence of a gateway
Commission	Yes	Tables, arithmetic, worked examples, boundaries
Salary, withdrawals, lending, capital, expenses, profit	Yes	
Reconciliation	Yes	Detection-only
Notifications	Yes	Push, two email systems, in-app
Configuration	Yes	Including how a change takes effect
Reporting	Yes	
Security and permissions	Yes	Including what a rule is actually worth
Audit	Yes	Including what is not audited
Edge cases	Yes	Six categories
Financial reconciliation risks	Yes	Fifteen traceability questions
Unknown features discovered	Yes	Nine, in §39
Missing features	Yes	Nine, in §38E
Gaps separated from decisions	Yes	43 entries, severity marked, business confirmation flagged
Open questions	Yes	28, grouped

What is NOT covered, and should be

Not covered	Why
The live configuration values	Reading production was not authorised. Every rate in this document is the documented fallback
Production data volumes and the real shape of the data	Same
Whether each described flow is actually used in practice	Code presence is not product usage, and this document says so where it can tell
A full API inventory, endpoint by endpoint	Routers and their business purpose are covered; a parameter-level inventory was not built
Per-screen request and response payloads	Screens are documented by purpose, actions and gating
Android compiled and run	No build environment was available
The security rules compiled	GAP-S001

PHASE 0 DOCUMENTATION COMPLETE — THIS IS THE CURRENT UNDERSTANDING OF ZIPPER

This document is **not** a statement that ZIPPER is complete.

What we know: the shape of the product, every role and its lifecycle, every major flow, the arithmetic of the money, what runs automatically, and what is recorded.

What was discovered: nine subsystems or behaviours the business had not described, the largest being a complete games control plane that nothing calls.

What is unclear: twenty-eight questions that the code cannot answer.

What appears missing: nine business requirements with no implementation, including the payment gateway, host transfer, the agency-suspension cascade, and any way to hold a payout.

What should be investigated next: the ten steps in §43, beginning with reading the live configuration — because every financial figure in this document is a documented fallback, not a live value.

Appendix A — Document Provenance

Every statement in this document was derived from the source of the four codebases at the state described below. No production data was read; no configuration document was fetched; no system was modified in producing it.

Source	Scope
`zipper-app`	All screens, repositories, view models, navigation, utilities
`zipper-crm`	All routes, libraries, components, the authentication provider and layout
`functions`	All source modules and the deployed function index
`backend`	All routers, services, core modules, the data layer and the test suites
`firestore.rules`, `storage.rules`	In full

Where a quotation appears in italics, it is the codebase’s own words, reproduced because a paraphrase would have been weaker evidence than the original.

Where this document says **NOT FOUND**, an exhaustive search was performed across all four codebases, excluding build output and vendored dependencies, and returned nothing.

Where this document says **UNKNOWN**, the question was pursued and the code does not answer it.